

[ÉTABLISSEMENT — à compléter]

[Département / UFR — à compléter]

MASTER 1

Systemes, reseaux, telecommunications et cybersécurité

[Intitulé exact de la formation — à compléter]

MÉMOIRE

THÈME

Conception d'une architecture de communication temps réel sécurisée et résiliente pour les applications multimédias distribuées

Cas d'étude : le module Messagerie du prototype expérimental Allyza



Présenté par : [Nom et prénom — à compléter]

Sous l'encadrement de : [Encadreur(s) — à compléter]

Année académique : [à compléter]

Remerciements

[Page à rédiger par l'auteur : remerciements à l'encadreur, à l'établissement, aux relecteurs et à l'entourage. Aucun nom n'est inséré ici afin de ne rien présumer.]

Résumé

Ce mémoire étudie la conception d'une architecture de communication temps réel sécurisée et résiliente pour des applications multimédias distribuées. Le support d'étude est Allyza, une application web progressive (PWA) privée destinée à deux utilisateurs, dont seul le module de messagerie est traité : messages textuels, stickers, réactions, photographies, messages vocaux, appels audio et vidéo WebRTC, notifications et verrouillage de l'application.

Une phase d'audit du dépôt source (34 commits, du 21 au 25 septembre 2026) a permis de reconstituer l'architecture réellement présente : un client Next.js/React, des actions serveur déployées sur Vercel, une base PostgreSQL protégée par des politiques de sécurité au niveau des lignes (RLS) et un stockage objet privé fournis par Supabase, un canal temps réel (postgres_changes et diffusion privée) servant aussi de signalisation WebRTC, et un service de relais TURN de Cloudflare dont seuls des identifiants éphémères sont utilisés. Les fonctionnalités sont classées en quatre catégories (implémentée, partielle, prévue, proposition) afin de ne jamais présenter comme acquis ce qui relève de la perspective.

Sur le plan expérimental, seules des mesures de calcul et des tests automatisés ont été réalisés : le coût d'un chiffrement de bout en bout optionnel (AES-GCM, ECDH P-256) est inférieur à 1,2 ms en médiane pour un message de 4 000 caractères et de l'ordre de 19 ms pour un fichier de 3 Mo sur le poste de test, et 96 contrôles d'isolation en base ainsi que 22 contrôles cryptographiques sont réussis. Les scénarios réseau (latence, perte, coupure, changement de réseau) font l'objet d'un protocole détaillé, mais aucun résultat n'est revendiqué à ce stade. Le mémoire conclut sur les compromis entre sécurité, performance et disponibilité et sur les limites propres au contexte PWA, et propose une architecture cible (observabilité, E2EE à cliquet, relais redondants, application native) présentée strictement comme perspective.

Mots-clés : communication temps réel ; WebRTC ; messagerie ; cybersécurité ; chiffrement de bout en bout ; résilience ; réseaux hétérogènes ; PWA ; observabilité ; Row Level Security.

Abstract

This thesis studies the design of a secure and resilient real-time communication architecture for distributed multimedia applications. The case study is Allyza, a private progressive web app (PWA) for two users; only its messaging module is examined: text, stickers, reactions, photos, voice notes, WebRTC audio and video calls, push notifications and application lock.

An audit of the source repository (34 commits, 21–25 September 2026) reconstructs the architecture actually present: a Next.js/React client, server actions hosted on Vercel, a PostgreSQL database with row-level security and private object storage from Supabase, a real-time channel that also carries WebRTC signalling, and Cloudflare's TURN relay used only through short-lived credentials. Features are classified as implemented, partial, planned or proposed so that perspectives are never presented as results.

Only computational measurements and automated tests were carried out: the cost of an optional end-to-end encryption layer (AES-GCM, ECDH P-256) is below 1.2 ms (median) for a 4,000-character message and about 19 ms for a 3 MB file on the test workstation, and 96 database isolation checks plus 22 cryptographic checks pass. Network scenarios (latency, loss, outage, network change) are specified as an experimental protocol; no result is claimed. The thesis discusses trade-offs between security, performance and availability and the limits of the PWA context, and proposes a target architecture as future work.

Keywords: real-time communication; WebRTC; messaging; cybersecurity; end-to-end encryption; resilience; heterogeneous networks; PWA; observability; Row Level Security.

Liste des acronymes

Sigle	Signification
AAD	Additional Authenticated Data (données authentifiées additionnelles, AES-GCM)
AES-GCM	Advanced Encryption Standard en mode Galois/Counter
API	Application Programming Interface
CGNAT	Carrier-Grade NAT
DTLS	Datagram Transport Layer Security
E2EE	End-to-End Encryption, chiffrement de bout en bout
ECDH	Elliptic Curve Diffie–Hellman
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
ICE	Interactive Connectivity Establishment
JWT	JSON Web Token
MAC	Message Authentication Code
MITM	Man-in-the-Middle
NAT	Network Address Translation
PBKDF2	Password-Based Key Derivation Function 2
PWA	Progressive Web App
RLS	Row Level Security
RTP / SRTP	Real-time Transport Protocol / Secure RTP
SDP	Session Description Protocol
SFU	Selective Forwarding Unit
STUN	Session Traversal Utilities for NAT
TLS	Transport Layer Security
TOFU	Trust On First Use
TURN	Traversal Using Relays around NAT
VAPID	Voluntary Application Server Identification (Web Push)
WSS	WebSocket Secure

Table des matières

INTRODUCTION GÉNÉRALE.....	11
1. Contexte et justification.....	11
2. Problématique.....	11
3. Hypothèses de travail	11
4. Objectifs	12
5. Méthodologie.....	12
6. Délimitation du travail.....	12
7. Limites du travail.....	13
8. Organisation du mémoire	13
CHAPITRE 1 — CONTEXTE, ÉTAT DE L'ART ET CADRE CONCEPTUEL.....	14
1.1. Communications temps réel	14
1.2. Applications multimédias distribuées.....	14
1.3. Réseaux hétérogènes	14
1.4. Contraintes des communications temps réel	15
1.5. Technologies de communication temps réel	15
1.6. Sécurité des communications	15
1.7. Résilience des communications.....	16
1.8. Architectures distribuées et services temps réel	16
1.9. Solutions existantes	17
1.10. Synthèse de l'état de l'art	17
CHAPITRE 2 — PRÉSENTATION ET ANALYSE DU PROJET ALLYZA	18
2.1. Présentation générale d'Allyza.....	18
2.2. Vision et objectifs du projet	18
2.3. Fonctionnalités générales de la plateforme	18
2.4. Positionnement du module Messagerie	18
2.5. Besoins fonctionnels.....	18
2.6. Besoins non fonctionnels.....	19
2.7. Contraintes techniques et environnementales	19
2.8. Cas d'utilisation du module Messagerie.....	20
2.9. Scénarios de communication.....	20
2.10. Matrice de vérification : état réel des fonctionnalités.....	20
CHAPITRE 3 — CONCEPTION DE L'ARCHITECTURE DE COMMUNICATION.....	21

3.1. Principes architecturaux	21
3.2. Architecture générale d'Allyza (actuelle)	21
3.3. Architecture du module Messagerie	22
3.4. Architecture de la signalisation	24
3.5. Architecture des communications audio et vidéo	24
3.6. Gestion des messages et des médias	25
3.7. Gestion de la persistance	26
3.8. Gestion des états de communication	26
3.9. Gestion des coupures et reconnections	27
3.10. Sécurisation de l'architecture	27
3.11. Résilience de l'architecture	28
3.12. Architecture actuelle et architecture cible	29
3.13. Diagrammes d'architecture	29
3.14. Diagrammes de séquence	30
3.15. Justification des choix technologiques	30
CHAPITRE 4 — IMPLÉMENTATION DU PROTOTYPE ALLYZA	32
4.1. Environnement de développement	32
4.2. Structure générale du prototype	32
4.3. Implémentation de la messagerie	32
4.4. Transmission des médias	33
4.5. Messages vocaux	33
4.6. Appels audio	33
4.7. Appels vidéo	33
4.8. Signalisation des appels	33
4.9. Persistance des données	34
4.10. Gestion des reconnections	34
4.11. Gestion des notifications	34
4.12. Mesures de sécurité mises en œuvre : chiffrement de bout en bout	34
4.13. Limites de l'implémentation	35
4.14. Problèmes techniques rencontrés et corrections	35
CHAPITRE 5 — EXPÉRIMENTATION ET ÉVALUATION	40
5.1. Méthodologie expérimentale	40
5.2. Environnement de test	40
5.3. Métriques et instrumentation proposées	40

5.4. Scénarios de test	41
5.5. Évaluation de la sécurité.....	43
5.6. Évaluation de la résilience	44
5.7. Résultats obtenus	44
5.8. Analyse des résultats	46
CHAPITRE 6 — DISCUSSION, LIMITES ET PERSPECTIVES	47
6.1. Discussion des résultats	47
6.2. Adéquation entre les objectifs et les résultats.....	47
6.3. Compromis entre sécurité, performance et disponibilité.....	47
6.4. Apports du prototype	48
6.5. Limites actuelles	48
6.6. Perspectives d'amélioration.....	49
6.7. Perspectives de déploiement à plus grande échelle	49
CONCLUSION GÉNÉRALE	50
BIBLIOGRAPHIE	51
ANNEXES	53
Annexe A — Matrice de vérification des fonctionnalités	53
Annexe B — Variables d'environnement (noms uniquement)	55
Annexe C — Événements d'instrumentation proposés	55
Annexe D — Commandes de reproduction des mesures	55
Annexe E — Chronologie du dépôt	56

Liste des figures

Figure 1 — Architecture générale actuelle du module Messagerie.....	22
Figure 2 — Diagramme de séquence : envoi d'un message texte	23
Figure 3 — Diagramme de séquence : réception, accusé de lecture et rattrapage	23
Figure 4 — Diagramme de séquence : établissement d'un appel audio ou vidéo	24
Figure 5 — Diagramme de séquence : envoi d'une image ou d'un message vocal	25
Figure 6 — Machine à états d'un appel et séparation CallProvider / CallOverlay	26
Figure 7 — Cycle de résilience d'un appel	27
Figure 8 — Couches de sécurité de l'architecture actuelle	27
Figure 9 — Architecture cible proposée (perspective).....	29
Figure 10 — Diagramme de séquence : appel entrant et sonnerie	30
Figure 11 — Coût de calcul du chiffrement (mesures réelles, médiane et p95)	45
Figure 12 — Surcoût de taille d'un texte chiffré (base64, IV et étiquette inclus).....	45
Figure 13 — Triangle des compromis sécurité – performance – disponibilité	47

Liste des tableaux

Tableau 1 — Grandeurs caractéristiques d'une communication temps réel	15
Tableau 2 — Besoins fonctionnels du module Messagerie.....	18
Tableau 3 — Besoins non fonctionnels.....	19
Tableau 4 — Cas d'utilisation principaux.....	20
Tableau 5 — Synthèse de la classification des fonctionnalités.....	20
Tableau 6 — Tables et ressources de la messagerie.....	22
Tableau 7 — Mécanismes de sécurité : état de la preuve.....	28
Tableau 8 — Écarts entre architecture actuelle et architecture cible	29
Tableau 9 — Choix technologiques, motifs et alternatives.....	30
Tableau 10 — Organisation du code de la messagerie.....	32
Tableau 11 — Environnements : réalisé et prévu	40
Tableau 12 — Métriques	40
Tableau 13 — Coût de calcul de l'E2EE (ms, médiane / 95e percentile)	44
Tableau 14 — Surcoût de taille du chiffrement	45
Tableau 15 — Autres vérifications automatiques	45
Tableau 16 — Objectifs et degré d'atteinte.....	47
Tableau 17 — Analyse des compromis.....	48
Tableau 18 — Matrice de vérification : élément, classe, preuve dans le projet, technologie, limites	53
Tableau 19 — Variables d'environnement utilisées par la messagerie.....	55
Tableau 20 — Événements proposés pour l'observabilité des appels et des médias	55
Tableau 21 — Jalons du dépôt (34 commits entre le 21 et le 25 septembre 2026).....	56

Les numéros de page de ces listes sont calculés par Word : à l'ouverture, accepter la mise à jour des champs (ou Ctrl+A puis F9).

INTRODUCTION GÉNÉRALE

1. Contexte et justification

Les échanges numériques reposent de plus en plus sur des services qui combinent texte, images, voix et vidéo dans une même interface. Ces services doivent fonctionner sur des accès très différents (Wi-Fi domestique, 4G, 5G, réseaux d'entreprise filtrés), derrière des traducteurs d'adresses (NAT) et avec des conditions de transmission variables dans le temps. La communication temps réel se distingue du transfert de fichiers par sa tolérance limitée au retard : la recommandation ITU-T G.114 considère qu'un délai de transmission unidirectionnel de l'ordre de 150 ms est acceptable pour la plupart des usages conversationnels [24]. Elle se distingue aussi par la nature de ses données : une conversation privée contient des informations personnelles dont la compromission est difficilement réversible.

Concevoir une telle plateforme n'est donc pas un simple exercice de développement d'interface. Il faut arbitrer entre des propriétés qui peuvent s'opposer : chiffrer davantage ajoute du calcul et de la gestion de clés, relayer les flux pour traverser les NAT ajoute un saut réseau, reconnecter automatiquement ajoute de l'état à gérer et à tester, et protéger les métadonnées limite les informations disponibles pour diagnostiquer. Ces arbitrages sont rarement documentés pour des prototypes de petite taille, alors qu'ils déterminent la qualité réelle de la communication.

Allyza, application web progressive privée développée entre le 21 et le 25 septembre 2026, offre un terrain d'étude adapté : son module de messagerie réunit en un seul système des échanges transactionnels (texte, réactions), des médias différés (photos, messages vocaux) et des flux temps réel (appels audio et vidéo), le tout contraint par l'environnement d'une PWA sur téléphone.

2. Problématique

Question centrale. Comment concevoir une architecture permettant de garantir simultanément confidentialité, intégrité, disponibilité et faible latence dans une plateforme de communication multimédia temps réel fonctionnant sur des réseaux hétérogènes ?

Cette question se décline en sous-questions : (i) comment protéger la confidentialité et l'intégrité des contenus et de la signalisation sans dégrader sensiblement la latence perçue ; (ii) comment maintenir la continuité d'une communication lorsque le réseau ou le cycle de vie de l'application la perturbe ; (iii) quelles propriétés peuvent être démontrées par mesure et lesquelles ne peuvent, à ce stade, qu'être argumentées.

3. Hypothèses de travail

- **H1** — Une architecture qui sépare l'état d'une communication de son affichage est plus tolérante aux changements de contexte de l'interface (fermeture de l'écran d'appel, passage en arrière-plan) qu'une architecture où l'écran porte l'état.
- **H2** — Le coût de calcul d'un chiffrement applicatif de bout en bout est faible devant les délais réseau, pour des messages et des fichiers de taille courante. *Cette hypothèse est partiellement*

testée par mesure de calcul (chapitre 5) ; l'impact sur la latence de bout en bout n'est pas mesuré.

- **H3** — Un relais TURN est nécessaire pour établir de façon fiable des appels entre réseaux différents ; sans lui, le taux d'échec augmente. *Hypothèse issue de la littérature [3][5], non mesurée sur Allyza.*
- **H4** — Les mécanismes de sécurité en transit (TLS, DTLS-SRTP) ne suffisent pas à protéger les contenus contre l'opérateur du service ; seul un chiffrement de bout en bout le permet [7][28].

4. Objectifs

Objectif général. Concevoir et étudier une architecture de communication temps réel sécurisée et résiliente pour une application multimédia distribuée, à travers le cas d'étude Allyza.

Objectifs spécifiques.

- reconstituer, à partir du code, l'architecture réellement implémentée du module de messagerie et la distinguer d'une architecture cible ;
- analyser les mécanismes de signalisation, de négociation et de reconnexion des appels audio et vidéo ;
- étudier la gestion des médias (photos, messages vocaux) et la persistance des échanges ;
- analyser le modèle de sécurité (authentification, autorisation, stockage privé, chiffrement en transit, chiffrement de bout en bout optionnel) et ses limites, y compris pour les métadonnées ;
- documenter les problèmes techniques réellement rencontrés, leurs causes, leurs corrections et ce qui reste à valider ;
- construire une méthodologie expérimentale reproductible et rendre compte honnêtement des mesures déjà disponibles.

5. Méthodologie

La démarche suit sept étapes : analyse du besoin, état de l'art, spécification, conception, implémentation, expérimentation et analyse. L'audit du dépôt a été réalisé par lecture du code source, des migrations SQL, de la configuration et de l'historique Git ; il a été complété par l'exécution des suites de tests automatisées du projet et d'un micro-banc d'essai des primitives cryptographiques. Chaque affirmation sur Allyza renvoie à un élément vérifiable du dépôt ; lorsque la vérification n'est pas possible, la mention « Information à vérifier dans l'implémentation actuelle » est utilisée.

Dans tout le mémoire, on distingue explicitement : le **fait observé** (présent dans le code ou dans l'historique), le **résultat expérimental** (mesuré), l'**analyse**, l'**hypothèse**, la **proposition** et la **perspective**. Les fonctionnalités sont classées en : **A** réellement implémentée ; **B** partiellement implémentée ; **C** prévue mais non finalisée ; **D** proposition ou perspective.

6. Délimitation du travail

Le travail porte sur le module Messagerie d'Allyza : messages texte, stickers, émojis, réactions, édition, suppression, accusés de lecture, photographies, messages vocaux, appels audio et vidéo, signalisation, persistance des appels, notifications et verrouillage de l'application. Les autres

espaces d'Allyza (suivi personnel, journal, souvenirs, jeux, etc.) ne sont évoqués que lorsqu'ils influencent l'architecture de communication.

7. Limites du travail

- Aucune campagne de mesure en conditions réseau réelles ou dégradées n'a été réalisée : le chapitre 5 fournit le protocole, pas des résultats de terrain.
- Les tests ont été exécutés sur un poste de développement (Windows, Node.js v26.1.0) et dans une base PostgreSQL en mémoire (PGlite), non sur des téléphones ni sur l'infrastructure de production.
- L'état de configuration de la production (variables d'environnement, application de la migration de chiffrement) n'est pas vérifiable depuis le dépôt.
- Les références bibliographiques ont été citées de mémoire à partir de sources normatives et académiques connues ; elles doivent être revérifiées avant dépôt (voir bibliographie).

8. Organisation du mémoire

Le chapitre 1 présente le contexte, l'état de l'art et le cadre conceptuel. Le chapitre 2 analyse le projet Allyza, ses besoins et la matrice de vérification des fonctionnalités. Le chapitre 3 expose la conception de l'architecture de communication, actuelle puis cible. Le chapitre 4 décrit l'implémentation du prototype et les problèmes rencontrés. Le chapitre 5 présente la méthodologie expérimentale et les mesures effectivement disponibles. Le chapitre 6 discute les résultats, les limites et les perspectives, avant la conclusion générale.

CHAPITRE 1 — CONTEXTE, ÉTAT DE L'ART ET CADRE CONCEPTUEL

Ce chapitre rassemble les notions nécessaires pour analyser une architecture de communication multimédia : nature des contraintes temps réel, technologies de transport et de signalisation, modèle de sécurité des communications, résilience et observabilité. Il ne décrit pas Allyza, traité à partir du chapitre 2.

1.1. Communications temps réel

Une communication est dite temps réel lorsque la valeur de l'information dépend du délai avec lequel elle est délivrée. La voix et la vidéo interactives sont des flux inélastiques : un paquet arrivé trop tard est inutile, alors qu'un transfert de fichier tolère de longs délais et se retransmet. Cette distinction explique que les protocoles de média temps réel privilégient la ponctualité sur la fiabilité complète : RTP transporte les échantillons avec horodatage et numéro de séquence mais sans retransmission garantie [11], et SRTP en assure la confidentialité et l'intégrité [10].

1.2. Applications multimédias distribuées

Une application multimédia distribuée associe des clients hétérogènes, des services de coordination et des ressources de stockage ou de relais. On y distingue généralement trois familles de trafic, qui n'ont pas les mêmes exigences : (i) les **données transactionnelles** (messages, réactions, états de lecture), qui exigent cohérence et persistance mais tolèrent quelques centaines de millisecondes ; (ii) les **médias différés** (photographies, messages vocaux enregistrés), dominés par le débit et l'espace de stockage ; (iii) les **flux temps réel** (appels), dominés par la latence, la gigue et la perte. Un même système doit donc composer trois logiques de qualité de service (voir [32]).

1.3. Réseaux hétérogènes

Un utilisateur mobile passe d'un réseau Wi-Fi à un réseau cellulaire sans que l'application ne l'ait décidé ; son adresse IP publique change alors, et les associations établies dans les NAT sont perdues. Les NAT (y compris les NAT de niveau opérateur, CGNAT) empêchent l'ouverture de connexions entrantes et rendent la connectivité directe entre deux pairs incertaine [3][36]. Le débit disponible, le délai et le taux de perte varient dans le temps ; la connectivité peut être interrompue sans signal explicite. Une architecture temps réel doit donc être conçue pour un environnement **non déterministe** : elle ne peut pas supposer qu'une connexion établie le restera, ni qu'un événement émis sera reçu.

1.4. Contraintes des communications temps réel

Tableau 1 — Grandeurs caractéristiques d'une communication temps réel

Grandeur	Définition	Effet sur l'usage
Latence (délai)	Temps de transmission d'un paquet ou d'un message entre deux points	Interactivité ; au-delà d'environ 150 ms unidirectionnel, la conversation se dégrade [24]
Gigue (jitter)	Variation du délai entre paquets successifs	Nécessite un tampon de gigue, qui ajoute du délai
Perte de paquets	Proportion de paquets non reçus ou reçus trop tard	Artefacts audio/vidéo ; correction par dissimulation ou retransmission
Débit	Quantité de données par unité de temps	Qualité vidéo ; adaptation du codage
Disponibilité	Aptitude du service à être rendu lorsqu'on le sollicite [27]	Réussite de l'établissement et continuité de l'appel

1.5. Technologies de communication temps réel

1.5.1. WebSocket et événements serveur

Le protocole WebSocket établit un canal bidirectionnel persistant au-dessus de HTTP [13]. Il permet de pousser des événements (nouveau message, indicateur de saisie) sans interrogation répétée. Il ne garantit pas la livraison après une coupure : un client déconnecté manque les événements émis pendant son absence, ce qui impose un mécanisme de rattrapage par lecture de l'état.

1.5.2. WebRTC

WebRTC désigne un ensemble de protocoles et d'interfaces de programmation permettant à des navigateurs d'échanger audio, vidéo et données en pair-à-pair [1][20][31]. La négociation d'une session repose sur l'échange de descriptions de session au format SDP [6] selon le modèle offre/réponse décrit par JSEP [2] ; **WebRTC ne normalise pas le canal de signalisation** qui transporte ces descriptions : chaque application choisit le sien. Les candidats de connectivité sont découverts et testés par ICE [3], qui s'appuie sur STUN pour découvrir l'adresse publique [4] et sur TURN pour relayer le trafic lorsque la connexion directe est impossible [5]. Les flux média sont protégés par SRTP, dont les clés sont établies par une négociation DTLS entre les deux pairs [9][10].

1.5.3. Signalisation et relais

Parce que la signalisation est hors norme, sa sécurité relève de l'application. L'architecture de sécurité de WebRTC suppose que le canal de signalisation transmette fidèlement les empreintes de certificats DTLS ; un acteur qui contrôle ce canal peut, en l'absence d'authentification supplémentaire des empreintes, s'interposer dans l'appel [7][8]. Cette observation est centrale pour la suite (section 3.10).

1.6. Sécurité des communications

On retient les trois propriétés classiques : **confidentialité** (seules les parties autorisées accèdent au contenu), **intégrité** (le contenu n'est pas modifié sans détection) et **disponibilité** (le service est

utilisable quand il le faut) [27]. S'y ajoutent l'authentification, l'autorisation et la non-répudiation selon le contexte. Saltzer et Schroeder rappellent des principes de conception qui guident l'analyse : moindre privilège, défense en profondeur, mécanisme économique, conception ouverte [25].

1.6.1. Chiffrement en transit et chiffrement de bout en bout

Le **chiffrement en transit** (TLS [12], DTLS [9]) protège le contenu entre deux extrémités d'une liaison : le serveur qui termine la connexion TLS voit le contenu en clair. Le **chiffrement de bout en bout (E2EE)** protège le contenu entre les terminaux des correspondants : le serveur ne manipule que du chiffré. Les deux ne sont pas interchangeables : un service entièrement protégé par TLS peut néanmoins lire, stocker et divulguer tous les messages [28]. C'est la distinction absolue retenue dans ce mémoire.

1.6.2. Primitives utilisées dans les messageries chiffrées

Les messageries chiffrées combinent un échange de clés (Diffie–Hellman sur courbes elliptiques [19]), une dérivation de clés (HKDF [15]) et un chiffrement authentifié (AES-GCM [18]). Les protocoles modernes ajoutent un cliquet de clés (Double Ratchet) qui procure le secret de transmission aval (forward secrecy) et la récupération après compromission [30][29]. Les messageries comparées dans la littérature (Signal, WhatsApp, Matrix) sont analysées par Unger et al. selon la confiance initiale, la conservation des conversations et la facilité d'usage [28].

1.6.3. Métadonnées

Un contenu chiffré n'implique pas des métadonnées protégées : l'identité des correspondants, l'horodatage, la fréquence, la durée des appels, la taille des messages et les adresses IP restent observables par l'infrastructure, et peuvent suffire à reconstituer des habitudes [28]. La protection des métadonnées est un problème distinct du chiffrement du contenu.

1.7. Résilience des communications

La résilience est ici entendue comme l'aptitude à maintenir ou rétablir un service acceptable malgré des fautes ou des perturbations [27]. Pour un appel, cela recouvre la détection de la perte de connectivité, la reprise de la négociation de chemin (le redémarrage d'ICE (ICE restart) est prévu par la spécification d'ICE [3]), la limitation du temps d'attente avant abandon honnête et la préservation de l'état local lorsque l'interface change. Pour la messagerie, cela recouvre la reprise après coupure par relecture de l'état manquant plutôt que par la seule confiance dans les événements.

1.8. Architectures distribuées et services temps réel

Un système distribué est confronté aux compromis décrits par le théorème CAP : en présence d'une partition réseau, on ne peut garantir à la fois la cohérence et la disponibilité [26]. Les interfaces de messagerie adoptent en pratique une **cohérence à terme** avec affichage optimiste : le message apparaît localement avant confirmation, puis est réconcilié avec l'état serveur [39]. L'observabilité (journaux, métriques, traces) conditionne la capacité à diagnostiquer et à mesurer un tel système [33][38].

1.9. Solutions existantes

Les solutions de communication temps réel se répartissent en trois familles. Les **services hébergés** (plateformes d'appel et de messagerie en ligne) délèguent la complexité à un fournisseur, au prix de la dépendance et, selon l'offre, d'un coût proportionnel à l'usage. Les **briques ouvertes** (serveurs SFU open source, serveurs TURN comme coturn, protocoles de messagerie fédérée) donnent la maîtrise mais demandent de l'exploitation. Enfin, les **messageries à chiffrement de bout en bout** reposent sur des protocoles publiés (Signal, Matrix) [30][29]. Le choix du prototype étudié (section 3.15) est situé par rapport à ces familles ; il ne s'agit pas d'une évaluation comparative de produits.

1.10. Synthèse de l'état de l'art

- WebRTC fournit un transport média sécurisé entre pairs, mais laisse à l'application la signalisation et donc une partie de la sécurité.
- La traversée de NAT exige STUN et, pour la fiabilité sur réseaux différents, un relais TURN [3][5].
- TLS et E2EE répondent à des menaces différentes ; les métadonnées restent exposées dans les deux cas.
- La résilience repose sur la détection, la reprise bornée dans le temps et la relecture d'état, plus que sur la confiance dans la livraison des événements.
- Sans instrumentation, les propriétés de latence et de disponibilité ne peuvent pas être démontrées.

CHAPITRE 2 — PRÉSENTATION ET ANALYSE DU PROJET ALLYZA

2.1. Présentation générale d'Allyza

Fait observé. Allyza est une application web progressive bilingue (français et anglais), installable sur l'écran d'accueil (manifeste avec affichage « standalone » et page de démarrage /home), destinée à un couple : deux comptes liés entre eux, dont les données sont isolées par appartenance au couple. Elle est développée avec Next.js 16.3.5 (routeur App Router et actions serveur), React 19.2.8, TypeScript 5 et Tailwind CSS 4, s'appuie sur Supabase (supabase-js 2.116, @supabase/ssr 0.12.7) pour la base, l'authentification, le stockage et le temps réel, et est déployée sur Vercel (information tirée des journaux de build fournis pendant le développement).

Le prototype est traité ici comme **terrain d'expérimentation** : il permet d'implémenter concrètement les mécanismes étudiés (signalisation, relais, chiffrement, reprise) et d'en observer les limites, sans prétendre qu'il constitue un produit de référence.

2.2. Vision et objectifs du projet

Le projet vise un espace numérique privé pour deux personnes, où la confidentialité prime sur la diffusion. Dans cette perspective, la messagerie n'est pas un service public à grande échelle : elle est une conversation unique entre les deux membres du couple (aucune conversation de groupe n'est présente dans le code). Cette restriction simplifie le modèle (un curseur de lecture par personne, des appels un-à-un) mais limite la portée des conclusions sur la scalabilité.

2.3. Fonctionnalités générales de la plateforme

Au-delà de la messagerie, Allyza comprend un suivi personnel, un journal partagé, des souvenirs, des jeux et un espace de détente. Ces modules ne sont pas étudiés. Ils partagent avec la messagerie l'authentification, le modèle de couple, le stockage privé et le système de notifications, qui constituent des dépendances de l'architecture étudiée.

2.4. Positionnement du module Messagerie

Le module Messagerie s'appuie sur le client PWA, l'application serveur, la base de données, le stockage privé, le canal temps réel, le service de relais TURN et le service Web Push. Leur agencement est représenté à la figure de la section 3.2.

2.5. Besoins fonctionnels

Tableau 2 — Besoins fonctionnels du module Messagerie

Réf.	Besoin	Classe
BF1	Échanger des messages texte (réponse à un message, édition, suppression, copie)	A
BF2	Réagir à un message par un émoji ou un sticker ; envoyer des stickers	A
BF3	Recevoir les messages en temps réel et après une période d'absence	A
BF4	Connaître l'état d'envoi/lecture et voir l'indicateur de saisie	A (lecture) ; D (livraison)
BF5	Envoyer et consulter des photographies, les enregistrer	A

Réf.	Besoin	Classe
BF6	Enregistrer et écouter des messages vocaux	A
BF7	Passer et recevoir des appels audio et vidéo avec sonnerie, micro, caméra, haut-parleur	A / B
BF8	Poursuivre l'appel hors de l'écran d'appel (palette réduite)	A (écran) ; B (arrière-plan)
BF9	Être notifié hors de l'application (messages, appels, appels manqués)	B
BF10	Consulter l'historique des appels et rappeler	A
BF11	Rechercher dans la conversation ; ouvrir les liens	A
BF12	Échanger des documents et fichiers quelconques	D (non implémenté)

2.6. Besoins non fonctionnels

Tableau 3 — Besoins non fonctionnels

Réf.	Exigence	Mécanisme visé dans le prototype
BN1	Confidentialité du contenu	RLS, stockage privé, URL signées ; E2EE optionnel
BN2	Intégrité	Chiffrement authentifié AES-GCM avec contexte lié ; triggers de garde en base
BN3	Disponibilité de l'appel	Relais TURN, redémarrage ICE, reprise du micro
BN4	Faible latence d'établissement	Mise en cache des serveurs ICE, offre diffusée sans attendre la base
BN5	Résilience aux changements d'interface	Séparation état d'appel / écran, Wake Lock
BN6	Persistance	Historique en base (messages, appels, curseurs de lecture)
BN7	Ergonomie mobile	Cadre calé sur le viewport visuel, retours haptiques, tonalités
BN8	Minimisation des fuites par notification	Charge utile limitée au type et au nom, sans contenu

2.7. Contraintes techniques et environnementales

- **PWA** : pas d'accès aux interfaces d'appel natives (CallKit/PushKit sur iOS) ; l'audio et le micro en arrière-plan dépendent du système ; l'audio web est verrouillé avant un premier geste de l'utilisateur.
- **Navigateurs** : la sélection de la sortie audio (setSinkId) n'est pas disponible sur Safari iOS ; la prise en charge des API (Wake Lock, Web Push, WebAuthn) varie.
- **Serverless** : les actions s'exécutent sur Vercel ; le limiteur de débit du code est en mémoire par instance (best-effort) ; les tâches non attendues peuvent être interrompues (d'où l'usage de after()).
- **Coût** : le choix de composants à offre gratuite ou à coût faible (Supabase, Vercel, TURN de Cloudflare) borne les capacités.
- **Deux utilisateurs** : les résultats de charge ne sont pas extrapolables.

2.8. Cas d'utilisation du module Messagerie

Tableau 4 — Cas d'utilisation principaux

Acteur	Cas d'utilisation	Préconditions
Membre du couple	Envoyer / recevoir un message, une photo, un vocal, un sticker	Authentifié ; couple complet (deux membres)
Membre du couple	Réagir, répondre, modifier, supprimer (ses propres messages)	Message existant
Appelant	Initier un appel audio ou vidéo ; annuler ; rappeler	Aucun autre appel actif
Appelé	Recevoir, accepter ou refuser un appel ; sonnerie	Application ouverte (écran d'appel) ou notification
Membre du couple	Quitter l'écran d'appel, réduire, rouvrir ; raccrocher	Appel en cours
Membre du couple	Activer le chiffrement de bout en bout, comparer le code de sécurité	Mot de passe de chiffrement choisi
Système	Rattraper les messages manqués ; notifier hors application	Reprise de visibilité / reconnexion

2.9. Scénarios de communication

Les scénarios retenus pour l'étude sont : message texte ; envoi de média ; message vocal ; appel audio ; appel vidéo ; appel entrant ; interruption réseau ; reconnexion ; réception d'un message pendant l'utilisation de l'application ou après une période d'arrière-plan. Ils structurent les diagrammes du chapitre 3 et le protocole du chapitre 5.

2.10. Matrice de vérification : état réel des fonctionnalités

La matrice ci-dessous résume la classification A/B/C/D. Le détail avec les preuves dans le dépôt figure en annexe A. Elle constitue le fondement du principe directeur du mémoire : en cas de doute entre ce qui serait techniquement logique et ce qui est présent dans le code, c'est le second qui est retenu.

Tableau 5 — Synthèse de la classification des fonctionnalités

Classe	Signification	Éléments principaux
A	Réellement implémentée (présente dans le code, relue)	Messages texte, stickers, réactions, édition/suppression, accusés de lecture, rattrapage, photos, vocaux, appels WebRTC, signalisation par diffusion, relais TURN via identifiants, sonnerie, historique, palette réduite, Web Push, verrouillage, RLS, URL signées
B	Partiellement implémentée ou non validée de bout en bout	E2EE (code et tests, validation entre deux comptes non réalisée, migration non confirmée en production) ; appels sur réseaux hétérogènes (non mesurés) ; haut-parleur (selon navigateur) ; persistance de l'appel en arrière-plan sur iOS ; appel entrant application fermée
C	Prévue, non finalisée	Aucune fonctionnalité identifiée dans cette catégorie à partir des éléments disponibles
D	Proposition / perspective	Fichiers et documents quelconques ; état « livré » distinct de « lu » ; observabilité ; CallKit/PushKit ; E2EE à cliquet ; architecture cible

CHAPITRE 3 — CONCEPTION DE L'ARCHITECTURE DE COMMUNICATION

Ce chapitre expose l'architecture **telle qu'elle est implémentée** (architecture actuelle) puis, dans la section 3.12, une architecture **cible** proposée. Les diagrammes de séquence représentent le comportement observé dans le code ; les éléments relevant de la proposition sont étiquetés comme tels.

3.1. Principes architecturaux

- **Défense en profondeur** [25] : l'accès aux données est contrôlé à plusieurs niveaux indépendants (authentification, politiques RLS, validation côté serveur, stockage privé), de sorte que l'échec d'une couche ne suffise pas à exposer le contenu.
- **Autorité côté base** : les règles d'appartenance au couple sont exprimées en SQL (RLS) et non seulement dans l'interface ; une interface modifiée ne peut pas les contourner.
- **Séparation de l'état d'une communication et de son affichage** (hypothèse H1) : l'appel vit dans un composant monté au niveau du layout, l'écran n'en est qu'une vue.
- **Affichage optimiste puis réconciliation** : un message est montré immédiatement, puis remplacé par la ligne confirmée ; la reprise après absence se fait par relecture de la base.
- **Minimisation du contenu hors du canal protégé** : les notifications ne portent ni texte ni média ; la signalisation ne porte que ce qui est nécessaire à la négociation.
- **Dégradation gracieuse** : les fonctions indisponibles (son verrouillé, sélection de sortie audio, relais non configuré) sont signalées ou masquées plutôt que simulées.

3.2. Architecture générale d'Allyza (actuelle)

Fait observé. L'architecture est une architecture de type « backend géré » : l'application serveur Next.js (actions serveur, proxy d'authentification) est exécutée sur Vercel ; Supabase fournit la base PostgreSQL, l'authentification, le stockage et deux mécanismes temps réel ; le navigateur de chaque utilisateur porte la quasi-totalité de la logique de présentation et de la logique d'appel. Il n'y a pas de serveur de signalisation dédié ni de serveur média propre : la signalisation transite par le canal de diffusion de Supabase et le média circule en pair-à-pair, éventuellement relayé par le service TURN de Cloudflare.

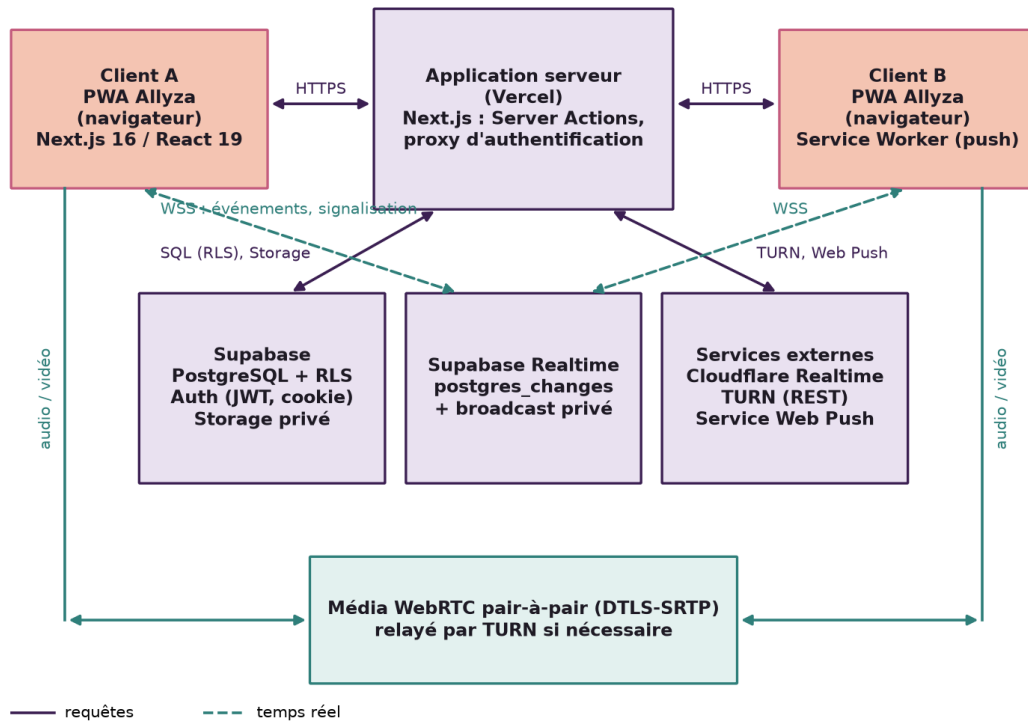


Figure 1 — Architecture générale actuelle du module Messagerie

3.3. Architecture du module Messagerie

Le modèle de données de la messagerie est volontairement compact. **Fait observé** (migrations 0003, 0005, 0009, 0010) :

Tableau 6 — Tables et ressources de la messagerie

Ressource	Rôle	Contraintes notables
messages	Messages texte, image, audio, sticker	kind ∈ {text, image, audio, sticker} ; body ≤ 4 000 car. (≤ 12 000 après 0010) ; storage_path unique et préfixé par <couple>/chat/ ; reply_to ; edited_at ; deleted_at (suppression logique) ; enc ∈ {0,1} ; aucune politique DELETE
message_cursors	Curseur « lu jusqu'à » par personne	une ligne par utilisateur ; mise à jour par son titulaire uniquement
reactions	Réactions (émoji ou sticker « s:n ») par message	unicité (auteur, cible, émoji) ; émoji ≤ 8 caractères
notifications	Pastilles « nouveau » sans contenu	insertion révoquée aux clients ; créées par trigger ; lues/mises à jour par le destinataire
push_subscriptions	Abonnements Web Push	accès réservé au propriétaire
calls	Métadonnées des appels (jamais le média)	statut ∈ {ringing, accepted, rejected, cancelled, missed, ended, failed} ; garde sur les champs immuables
e2ee_keys	Clé publique et clé privée enveloppée par personne	lecture de sa propre ligne seulement ; clé du partenaire via une fonction ne retournant que la clé publique
couple-media (Storage)	Photos et vocaux	bucket privé, 10 Mo par objet, types MIME image et audio (+ octet-stream pour le chiffré)

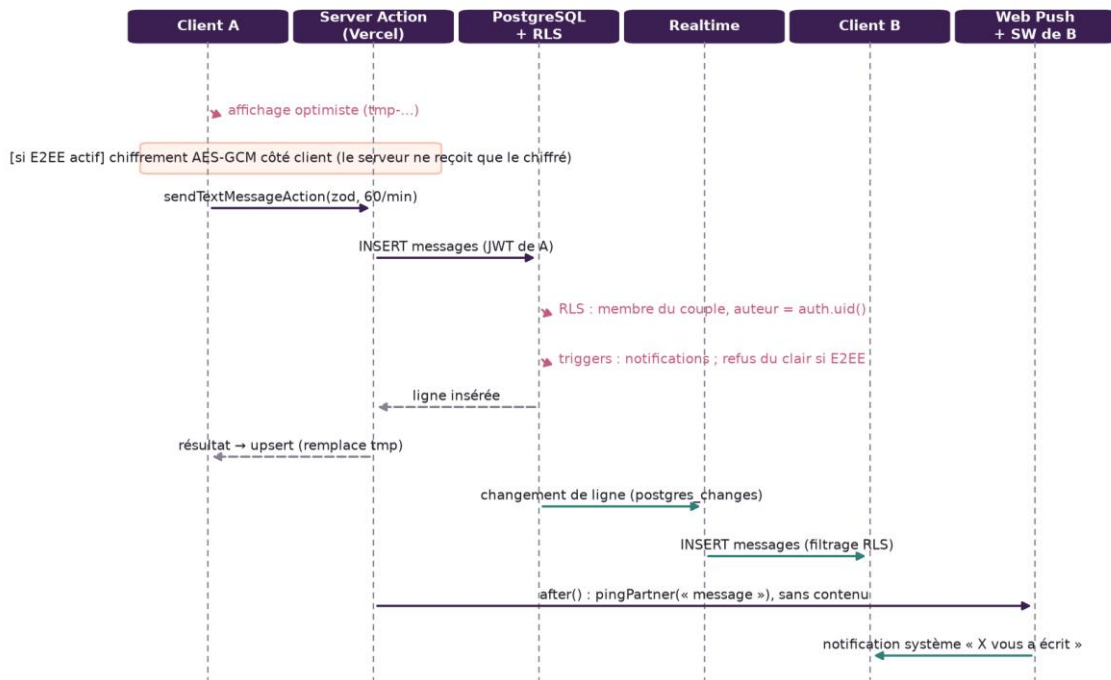


Figure 2 — Diagramme de séquence : envoi d'un message texte

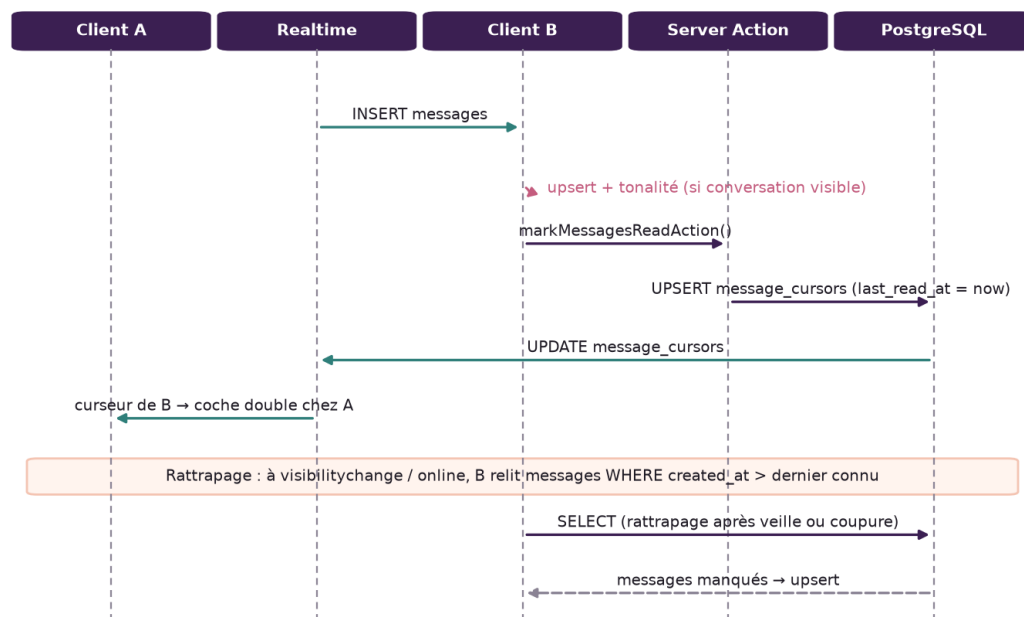


Figure 3 — Diagramme de séquence : réception, accusé de lecture et rattrapage

Analyse. Le chemin d'envoi comporte trois étapes serveur indépendantes : validation et limitation (action serveur), contrôle d'accès (RLS) et notification. Le message apparaît à l'expéditeur avant confirmation et chez le destinataire par l'événement de changement de ligne. L'accusé de lecture est un **curseur** et non un statut par message : plus simple et adapté à deux participants, mais il ne distingue pas l'état « livré » de l'état « lu ».

3.4. Architecture de la signalisation

Fait observé. La signalisation d'appel repose sur un canal de diffusion privé nommé `call:<identifiant du couple>`, protégé par des politiques sur `realtime.messages` qui n'autorisent que les membres du couple. Chaque signal est un objet `{callId, from, type, kind, sdp?, candidate?, muted?, cameraOff?, mac?}` avec `type` $\in$ `{ringing, offer, answer, ice, hangup, reject, media}`. La table `calls` complète ce canal : l'insertion d'une ligne (`postgres_changes`) fait sonner l'appelé même si le signal de diffusion est perdu, et la mise à jour de statut ferme l'appel de l'autre côté. Au démarrage et à chaque retour au premier plan, une lecture de l'appel actif (`myActiveCallAction`) rattrape un appel manqué par le canal.

Analyse. Cette double voie (diffusion non durable + table durable) est une réponse au caractère non fiable de la diffusion : la diffusion donne la rapidité, la table donne la garantie. Le prix est une logique de déduplication (ensemble d'identifiants d'appels clos, `closedIds`) pour éviter qu'une offre tardive ne fasse resonner un appel terminé.

3.5. Architecture des communications audio et vidéo

Fait observé. Les appels utilisent `RTCPeerConnection` avec `bundlePolicy` « max-bundle » et un réservoir de candidats (`iceCandidatePoolSize` = 2). La négociation suit le modèle offre/réponse : l'appelant crée l'offre ; tant que l'appel n'est pas décroché, l'offre est rediffusée toutes les 3 s (l'appelé peut ne pas encore être abonné) ; l'appelé répond après acceptation. Les candidats ICE sont échangés au fil de l'eau (trickle ICE) par le même canal et mis en file tant que la description distante n'est pas posée. Les serveurs ICE sont obtenus par une action serveur (voir 3.5.1) et mis en cache côté client pendant 2 h et préchargés à l'ouverture de l'application.

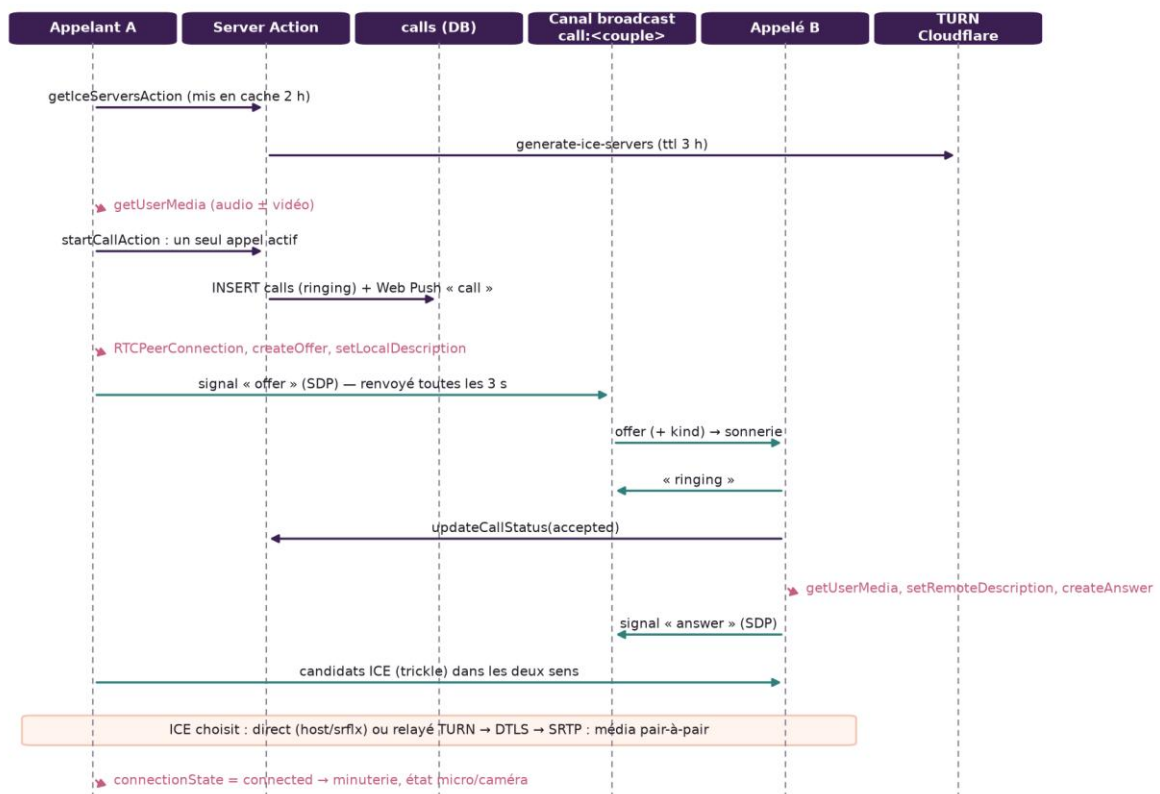


Figure 4 — Diagramme de séquence : établissement d'un appel audio ou vidéo

3.5.1. Infrastructure d'appel : STUN, TURN, Cloudflare

Fait observé. L'action `getIceServersAction` fournit, par ordre de priorité : (1) les identifiants du service **Cloudflare Realtime TURN**, obtenus par un appel REST authentifié (POST vers l'interface `generate-ice-servers` de Cloudflare, durée de vie 3 h), dont les URL en port 53 sont retirées ; (2) à défaut, un serveur **coturn** auto-hébergé avec identifiants éphémères calculés par HMAC à partir d'un secret partagé ; (3) à défaut, le serveur STUN public `stun.l.google.com:19302`. Aucun secret de longue durée n'atteint le navigateur.

Rôle exact de Cloudflare. Cloudflare n'intervient dans Allyza que comme **fournisseur d'un relais TURN** et d'identifiants éphémères. Il ne traite ni la signalisation, ni la messagerie, ni le stockage, ni l'hébergement de l'application. Le relais n'est sollicité que si ICE ne trouve pas de chemin direct ; les paquets qu'il relaie sont des paquets SRTP chiffrés de bout en bout entre les pairs. L'architecture ne constitue donc pas une architecture « cloud-native » complète ; elle utilise un service cloud ciblé. La présence effective de ces identifiants dans l'environnement de production : information à vérifier dans l'implémentation actuelle.

3.5.2. Contrôles et états média

Le micro et la caméra (API de capture [21]) sont contrôlés en activant ou désactivant la piste locale (`track.enabled`) et en signalant l'état à l'autre pair (signal « media »). L'inversion de caméra utilise un nouveau `getUserMedia` et `replaceTrack` sur l'émetteur vidéo. La sélection du haut-parleur utilise `setSinkId` sur l'élément média audible ; le bouton n'apparaît que si l'API existe et si au moins deux sorties sont énumérées. L'audio distant d'un appel audio est lu par un élément média invisible rattaché au flux distant.

3.6. Gestion des messages et des médias

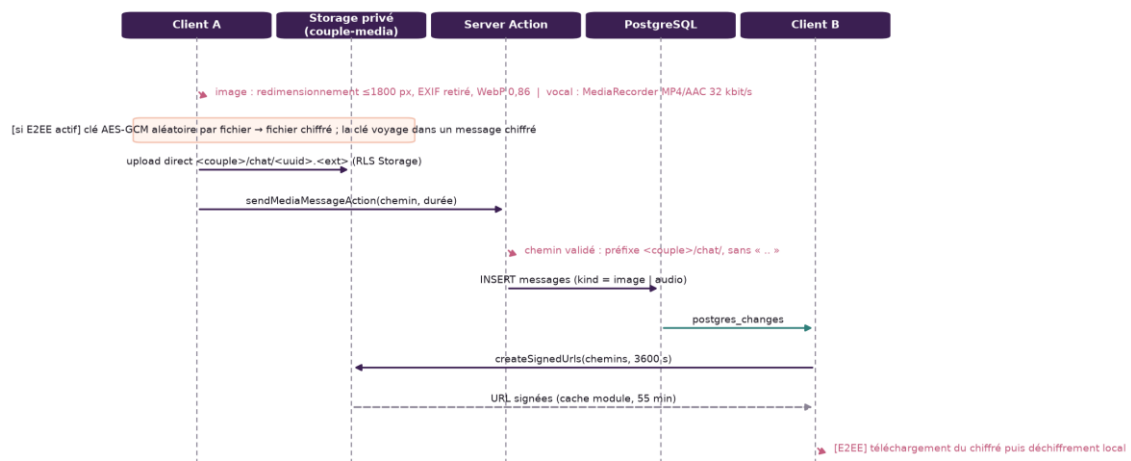


Figure 5 — Diagramme de séquence : envoi d'une image ou d'un message vocal

Fait observé. Les fichiers sont téléversés **directement** du navigateur vers le stockage privé, ce qui évite de les faire transiter par le serveur applicatif ; l'action serveur n'enregistre que la ligne de message après avoir vérifié que le chemin appartient au dossier du couple. Les images sont redimensionnées (plus grand côté $\leq 1\,800$ px), ré-encodées en WebP (qualité 0,86) par un passage par un canevas, ce qui supprime les métadonnées EXIF (position, appareil) ; les GIF sont

conservés. Les messages vocaux sont enregistrés avec MediaRecorder (types candidats dans l'ordre : audio/mp4, audio/aac, audio/webm;codecs=opus, audio/webm ; débit demandé 32 kbit/s). La lecture utilise des URL signées d'une heure, mises en cache côté client 55 minutes.

3.7. Gestion de la persistance

Les messages, réactions, curseurs, notifications et appels sont persistés en base. Les médias sont persistés dans le stockage objet. L'état d'une communication en cours (flux, connexion pair-à-pair, minuteries) n'est **pas** persisté : il vit en mémoire dans le navigateur. Le brouillon de message non envoyé est conservé localement (stockage du navigateur) et survit à un changement d'application ; il n'est pas chiffré au repos par l'application.

3.8. Gestion des états de communication

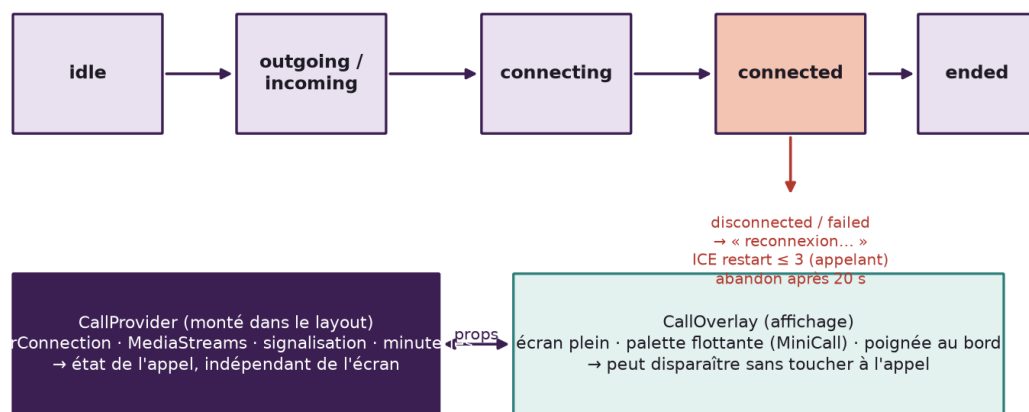


Figure 6 — Machine à états d'un appel et séparation CallProvider / CallOverlay

Fait observé. L'état d'appel est porté par le composant CallProvider, monté dans le layout de l'application, donc au-dessus de toutes les pages. Les phases sont : inactif, sortant ou entrant, connexion, connecté, terminé. L'écran (CallOverlay) peut être plein écran ou réduit en palette flottante (MiniCall) ; réduire ne touche ni aux pistes, ni à la connexion, ni à la signalisation, et le flux distant continue d'être lu par la palette elle-même. Le bouton « Retour » de l'écran d'appel réduit l'appel ; il ne le termine pas. Le raccrochage automatique à la fermeture de la page (événement pagehide) a été retiré : il se déclenchait sur le simple passage en arrière-plan sur iOS.

Analyse. Le traitement de la persistance de l'appel comme un problème de **gestion d'état** (et non d'interface) est le résultat de l'audit : il est la condition pour que l'utilisateur puisse naviguer dans l'application pendant un appel. Il ne résout pas ce que le système d'exploitation décide indépendamment de l'application (voir 6.5).

3.9. Gestion des coupures et reconnections

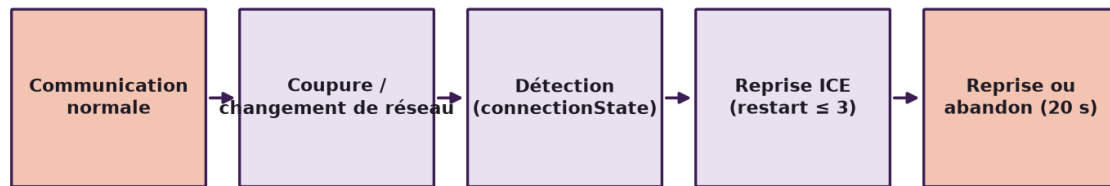


Figure 7 — Cycle de résilience d'un appel

Fait observé. Lorsque l'état de connexion devient « disconnected » ou « failed », l'interface affiche « reconnexion... » ; l'**appelant** relance la négociation avec un redémarrage d'ICE (jusqu'à 3 fois) en diffusant une nouvelle offre ; si rien ne rétablit la connexion en 20 s, l'appel se termine avec le statut « failed ». L'appelé conserve ses minuteries propres (30 s pour se connecter après réponse ; 60 s de sonnerie). Au retour au premier plan, si la piste micro locale est terminée par le système, elle est reprise par un nouveau getUserMedia et replaceTrack sur la même connexion ; un Wake Lock d'écran est demandé pendant l'appel connecté.

Limite. Aucune mesure du temps de reprise n'existe. Le mécanisme d'ICE restart est **implémenté** mais son efficacité lors d'un changement réel de réseau (Wi-Fi vers 4G) n'est pas démontrée (scénario 6, chapitre 5).

3.10. Sécurisation de l'architecture

Transport : HTTPS / WSS (TLS) assuré par les hébergeurs ; média WebRTC : DTLS-SRTP (navigateur)	classe P
Authentification : Supabase Auth (identifiant + mot de passe), JWT en cookie, vérifié par proxy.ts	classe A
Autorisation : RLS (is_couple_member), 70 politiques SQL ; Storage privé ; URL signées 1 h	classe A
Application : validation zod, débit limité (mémoire), chemins validés, clé service côté serveur	classe A
Verrou d'application : PIN / mot de passe (bcrypt N=16384) / biométrie WebAuthn, cookie signé 12 h	classe A
Notifications : charge utile sans contenu (type + nom), VAPID	classe A
Chiffrement de bout en bout (opt-in) : ECDH P-256 + HKDF → AES-GCM ; MAC des empreintes DTLS	classe B

A = implémenté, vérifié dans le code · B = dans le code, non validé de bout en bout · P = fourni par la plateforme, non audité ici

Figure 8 — Couches de sécurité de l'architecture actuelle

Tableau 7 — Mécanismes de sécurité : état de la preuve

Mécanisme	Description (fait observé)	Classe
Authentification	Supabase Auth, identifiant converti en adresse technique, mot de passe ; JWT [37] dans un cookie rafraîchi par le proxy, qui vérifie l'utilisateur à chaque requête ; limitation à 8 essais/15 min par identifiant et 20/15 min par IP	A
Autorisation	Fonction <code>is_couple_member(couple_id)</code> dans les politiques RLS de toutes les tables de la messagerie ; INSERT limité à <code>author_id = auth.uid()</code> ; pas de DELETE sur messages	A
Stockage	Bucket privé, politiques lecture/écriture par appartenance au couple, suppression réservée au propriétaire de l'objet ; URL signées d'une heure	A
Validation serveur	zod sur toutes les entrées ; chemins de médias vérifiés (préfixe, absence de « .. ») ; limites de débit (60 msg/min, 40 médias/min, 20 appels/min)	A
Secrets	Clé service Supabase et clé privée VAPID côté serveur uniquement ; identifiants TURN éphémères (3 h)	A
Verrou d'application	PIN/mot de passe haché par scrypt [17] (N=16 384, r=8, p=1), biométrie par clé publique WebAuthn vérifiée côté serveur, cookie HttpOnly SameSite=Lax signé de 12 h, rendu du seul écran de verrouillage quand verrouillé	A
Notifications	Charge utile : type, langue, nom affiché (surnom du destinataire), identifiant d'appel ; aucun contenu de message ; clés VAPID ; TTL 24 h (60 s pour un appel)	A
Chiffrement en transit	HTTPS/WSS fournis par Vercel et Supabase ; cookie Secure en production ; DTLS-SRTP des navigateurs pour le média. Configuration TLS des hébergeurs non audité ici	A (plateforme)
Chiffrement de bout en bout	Optionnel : voir 4.12. Code, tests unitaires et tests de base présents ; validation entre deux comptes non réalisée ; application de la migration en production non confirmée	B
Chiffrement au repos	Fourni par la plateforme d'hébergement des données (information à vérifier) ; aucun chiffrement applicatif des brouillons locaux	P

Distinction TLS / E2EE. Sans activation du chiffrement de bout en bout, l'exploitant de la base (Supabase), l'hébergeur applicatif et quiconque obtient la clé de service peuvent lire le contenu des messages et, via des URL signées, les médias. Le chiffrement TLS protège le transit, pas le contenu contre l'opérateur. C'est précisément la limite que la couche E2EE optionnelle tente de lever.

Menaces considérées (grille inspirée de [34]). Écoute réseau (TLS, DTLS-SRTP) ; usurpation d'identité (authentification) ; accès croisé entre couples (RLS) ; énumération et exposition de fichiers (bucket privé, URL signées expirant en une heure) ; force brute (limiteurs ; verrouillage temporaire de l'application après échecs) ; vol de session (cookies HttpOnly ; vérification du JWT par le proxy ; pas de révocation instantanée démontrée) ; abus d'API (validation et limites en mémoire par instance, donc contournables en environnement multi-instances) ; interposition dans la signalisation (MAC des empreintes DTLS, uniquement si l'E2EE est actif) ; jeu (liaison du MAC à l'identifiant d'appel et au rôle offre/réponse, mais sans contrôle de fraîcheur).

3.11. Résilience de l'architecture

La résilience repose sur cinq dispositifs observés : double voie de signalisation (diffusion + base), rattrapage des messages à la reprise de visibilité et au retour en ligne, redémarrage d'ICE, reprise du micro, et nettoyage des appels « zombies » (un appel « ringing » depuis plus de 90 s est marqué manqué ; un appel « accepted » depuis plus de 4 h est clos). Ces dispositifs sont des **moyens** ; leur **efficacité** n'est pas mesurée.

3.12. Architecture actuelle et architecture cible

Proposition / perspective, non confirmée comme implémentée. L'architecture cible reprend les mêmes responsabilités mais les sépare en services, ajoute l'observabilité et durcit la sécurité.

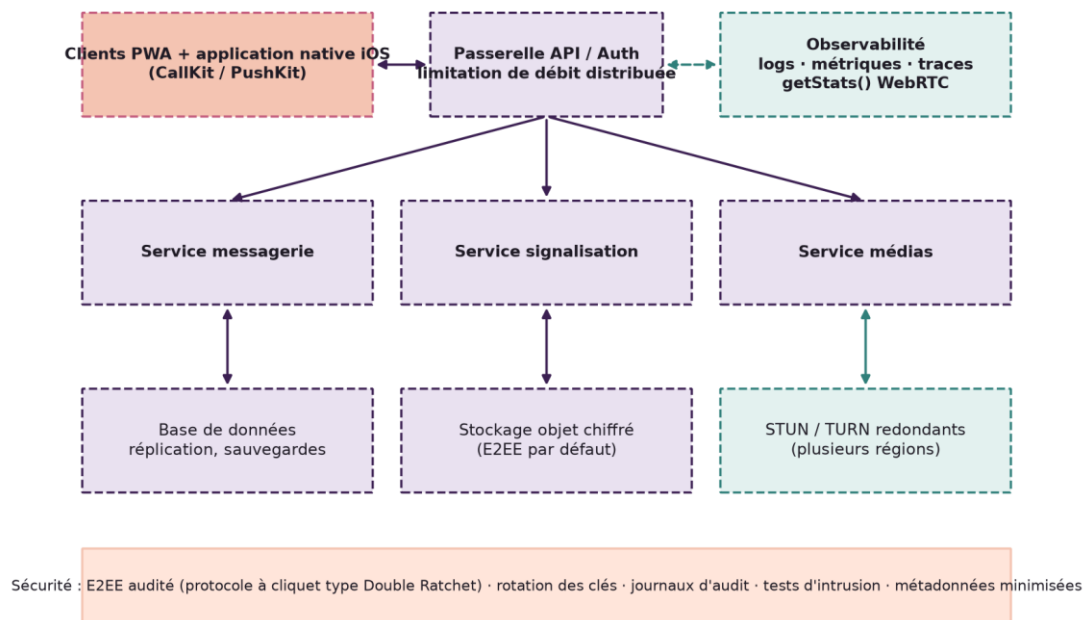


Figure 9 — Architecture cible proposée (perspective)

Tableau 8 — Écarts entre architecture actuelle et architecture cible

Domaine	Actuelle (implémentée)	Cible (proposition)
Services	Application monolithique Next.js + services gérés	Services messagerie, signalisation, médias derrière une passerelle
Limitation de débit	En mémoire par instance	Distribuée (stockage partagé)
E2EE	ECDH statique + AES-GCM, optionnel	Protocole à cliquet (type Double Ratchet) [30], rotation des clés, audit externe
Observabilité	<code>console.error/warn</code> uniquement	Journaux structurés, métriques, traces ; <code>getStats()</code> WebRTC collecté
Relais	Un fournisseur TURN (ou coturn) configuré par variables	Relais redondants multi-régions
Clients	PWA	PWA + application native iOS (CallKit, PushKit)
Fichiers	Images et audio	Documents et fichiers génériques chiffrés

3.13. Diagrammes d'architecture

Les diagrammes d'architecture figurent dans les sections 3.2 (architecture générale), 3.8 (états), 3.9 (résilience), 3.10 (sécurité) et 3.12 (architecture cible).

3.14. Diagrammes de séquence

Les diagrammes de séquence couvrent l'envoi et la réception d'un message (3.3), l'envoi d'un média (3.6), l'établissement d'un appel (3.5) et l'appel entrant avec sonnerie (ci-dessous). La reconnexion est représentée par le cycle de résilience (3.9). Le transfert d'un **document** n'est pas représenté : la fonctionnalité n'existe pas dans le code (classe D).

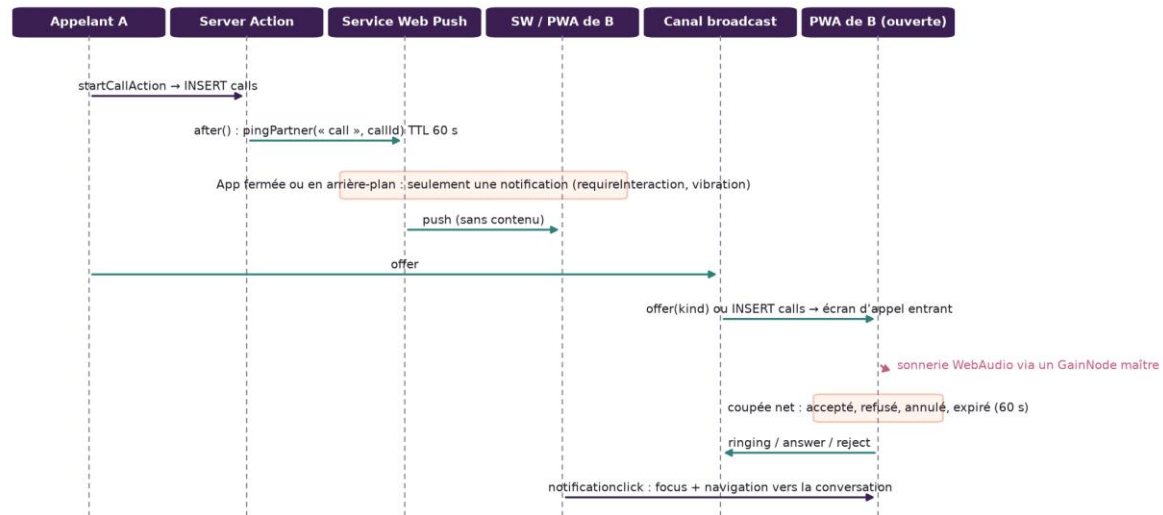


Figure 10 — Diagramme de séquence : appel entrant et sonnerie

La sonnerie est synthétisée avec la Web Audio API [23] : les oscillateurs passent par un unique nœud de gain maître, mis à zéro instantanément lorsque l'appel est accepté, refusé, annulé, expiré ou manqué, afin qu'aucun son ne subsiste. L'audio web étant verrouillé jusqu'à un premier geste de l'utilisateur, une application ouverte mais jamais touchée peut vibrer sans sonner ; hors de l'application, seule la notification est possible.

3.15. Justification des choix technologiques

Tableau 9 — Choix technologiques, motifs et alternatives

Choix	Motif (fait ou analyse)	Alternatives écartées ou possibles
PWA	Une base de code unique, installable, sans passage par une boutique	Application native (CallKit/PushKit) : proposition, hors périmètre actuel
Supabase (PostgreSQL + RLS + Realtime + Storage)	Autorisation exprimée en SQL au plus près des données ; temps réel intégré	API sur mesure + WebSocket propre : plus de contrôle, plus d'exploitation
Signalisation par diffusion Supabase	Pas de serveur à exploiter ; canaux privés soumis à la RLS	Serveur WebSocket dédié ; service de signalisation hébergé
WebRTC pair-à-pair	Média chiffré entre pairs sans serveur média ; adapté à deux participants	SFU/MCU pour les groupes (proposition)
TURN Cloudflare avec identifiants éphémères (coturn en secours)	Remplace un service d'appel payant ; aucun secret longue durée côté client	TURN auto-hébergé exclusivement ; services hébergés d'appel
Web Push (VAPID) avec after()	Seul canal de notification d'une PWA hors application ; after() évite l'interruption de la tâche sur serverless	Notifications natives APNs (proposition)
E2EE applicatif WebCrypto	Aucune dépendance ; primitives standard [22]	Protocole Signal/Matrix [29][30] : plus robuste, plus coûteux à implémenter

Choix	Motif (fait ou analyse)	Alternatives écartées ou possibles
Tonalités WebAudio synthétisées	Aucun fichier à télécharger ni à mettre en cache	Fichiers audio

CHAPITRE 4 — IMPLÉMENTATION DU PROTOTYPE ALLYZA

4.1. Environnement de développement

Le développement a été mené sous Windows 11 avec Node.js v26.1.0, Next.js 16.3.5 (mode développement sur le port 3100), TypeScript en mode strict, ESLint avec les règles du compilateur React, et déploiement sur Vercel. La base est un projet Supabase auquel les migrations SQL sont appliquées manuellement par l'éditeur SQL (information relevée pendant le développement). Le dépôt compte 34 commits entre le 21 et le 25 septembre 2026.

4.2. Structure générale du prototype

Tableau 10 — Organisation du code de la messagerie

Emplacement	Rôle
src/app/(app)/messages/chat/page.tsx	Page serveur : charge 150 derniers messages, réactions, curseurs, partenaire
src/features/messaging/ChatClient.tsx (≈ 790 lignes)	Conversation : état, temps réel, rattrapage, composer, médias, E2EE côté lecture/écriture
src/features/messaging/ (autres)	ChatShell (cadre sous clavier), MessageMenu (appui long), PhotoViewer, EmojiPicker, Linkified, CallHistory, ChatInfo
src/actions/messages.ts	Actions serveur : envoi texte/sticker/média, édition, suppression, lecture
src/features/calls/CallProvider.tsx (≈ 510 lignes), CallOverlay.tsx	État d'appel, WebRTC, signalisation ; affichage et palette
src/actions/calls.ts	Serveurs ICE, démarrage et statut d'appel, appel actif
src/lib/e2ee/crypto.ts, store.ts ; src/features/e2ee/E2eeProvider.tsx	Primitives, stockage de la clé, cycle de vie du chiffrement
src/lib/push.ts, public/sw.js	Envoi Web Push (serveur) ; Service Worker
src/lib/sfx.ts, src/features/calls/use-ring.ts	Tonalités et sonnerie
supabase/migrations/0001–0010	Schéma, RLS, déclencheurs, stockage, temps réel
scripts/dbtest.mjs, e2ee-test.mjs, e2ee-bench.mjs	Tests d'isolation SQL (PGlite), tests et banc de calcul cryptographiques

4.3. Implémentation de la messagerie

Réception et synchronisation. Le composant ChatClient s'abonne à un canal postgres_changes filtré par couple (INSERT et UPDATE de messages, UPDATE de message_cursors) et à un canal de diffusion privé pour l'indicateur de saisie (émis au plus toutes les 2,2 s). Un message est ajouté par une fonction upsert qui remplace le message temporaire (préfixe tmp-) correspondant, évitant les doublons dus à la double arrivée (réponse de l'action et événement). À chaque retour au premier plan ou retour en ligne, une requête relit les messages plus récents que le dernier connu (jusqu'à 100) : c'est le **rattrapage**, introduit après constat que les événements manqués pendant l'arrière-plan n'étaient jamais affichés.

Interactions. Appui long de 420 ms (annulé si le doigt bouge de plus de 10 px) ouvrant un menu contextuel positionné pour rester dans l'écran, avec un voile découpé (clip-path) pour ne pas flouter le message ; réactions rapides et sélecteur complet (20 stickers, groupes d'emojis) ; réponse avec citation ; édition et suppression logique réservées à l'auteur ; recherche accent-

insensible sur les textes déjà chargés ; liens http(s) cliquables ; séparateurs de jours (« Aujourd'hui », « Hier ») ; séparateur « Nouveaux messages » ; brouillon local.

États. Un message de l'utilisateur passe de « temporaire » (opacité réduite) à « envoyé » (ligne confirmée) ; le dernier message envoyé porte une coche, ou une double coche lorsque le curseur du partenaire l'a dépassé. Il n'y a pas d'état « livré » distinct (classe D). La pastille « nouveau » de la navigation provient de la table notifications, relue au retour au premier plan.

4.4. Transmission des médias

Les photographies sont sélectionnées par l'entrée de fichier du système (qui peut proposer l'appareil photo selon la plateforme : information à vérifier), limitées à six par envoi, traitées comme décrit en 3.6, regroupées à l'affichage lorsqu'elles proviennent du même auteur à moins de deux minutes d'écart (grille de deux colonnes, quatre tuiles au plus avec indication du reste) et ouvertes dans une visionneuse offrant l'enregistrement (feuille de partage lorsque disponible, sinon téléchargement). **Les documents et fichiers génériques ne sont pas pris en charge** (le sélecteur n'accepte que des images ; le bucket n'autorise que des types image et audio).

4.5. Messages vocaux

L'enregistreur ouvre le micro, choisit le premier type MIME pris en charge, enregistre par tranches de 100 ms, calcule un niveau sonore pour la forme d'onde et, à l'arrêt, assemble un Blob ; si le type fourni par MediaRecorder est vide (observé comme possible sur Safari), un type de repli est utilisé. Un enregistrement vide ou de moins de 200 octets n'est pas envoyé. L'extension du fichier (m4a, aac, webm) découle du type. À la réception, la lecture décode le fichier pour calculer une forme d'onde de 32 barres.

4.6. Appels audio

L'appel audio réutilise exactement la machine de l'appel vidéo sans piste vidéo. Le flux distant, sans élément visuel, est rattaché à un élément média invisible ; sans lui, aucun son n'était produit (voir 4.14). Les options de capture activent la suppression d'écho, la réduction de bruit et le contrôle automatique du gain. Le mute désactive la piste locale et signale l'état à l'autre pair.

4.7. Appels vidéo

La capture vidéo demande une résolution idéale de $1\,280 \times 720$ avec le mode de face choisi. L'interface comporte l'image distante plein écran, un aperçu local flottant déplaçable avec accrochage aux coins et permutation au toucher, et des contrôles qui se masquent après 4,5 s d'inactivité. La réduction de l'appel affiche une palette de 112×156 px, déplaçable, rangeable sur un bord avec une poignée de rappel.

4.8. Signalisation des appels

Voir 3.4 et 3.5. L'implémentation ajoute deux optimisations de latence : l'offre diffusée porte le type d'appel (kind) afin que le téléphone de l'appelé sonne **sans attendre** la réplification de la ligne de la base, et les serveurs ICE sont préchargés et mis en cache. L'effet de ces optimisations sur le temps d'établissement n'a pas été mesuré.

4.9. Persistance des données

Voir 3.7. La table calls enregistre statut, instants de début, de réponse et de fin : elle alimente l'historique des appels (résultat : durée, manqué, sans réponse, refusé, annulé, interrompu) et la fonction de rappel.

4.10. Gestion des reconnections

Voir 3.9. Les minuteriers observées sont : sonnerie côté appelant 45 s, côté appelé 60 s ; renvoi de l'offre 3 s ; garde de connexion après réponse 30 s ; abandon après déconnexion 20 s ; ICE restart ≤ 3 .

4.11. Gestion des notifications

Le Service Worker (version de cache « allyza-v5 ») ne met en cache que la coquille hors ligne et les ressources statiques immuables : pages, appels API, trafic Supabase et URL signées ne sont jamais mis en cache. À la réception d'un push, il affiche un texte localisé (« {nom} vous a écrit », « {nom} vous appelle », « Appel manqué de {nom} »), sauf pour un message lorsqu'une fenêtre visible est déjà sur la conversation. L'envoi (protocole Web Push [14]) côté serveur est exécuté dans after() de Next.js ; il lit les abonnements du destinataire avec la clé de service, respecte sa préférence de notification, supprime les abonnements expirés (404/410) et applique un TTL de 24 h (60 s pour un appel).

4.12. Mesures de sécurité mises en œuvre : chiffrement de bout en bout

Statut : classe B. Le chiffrement de bout en bout est **implémenté dans le code et testé unitairement**, mais (i) il n'a pas été validé entre deux comptes réels, (ii) la migration 0010 n'est pas confirmée comme appliquée en production, (iii) il est optionnel et n'est actif que lorsque les deux personnes l'ont activé.

- **Identité** : une paire ECDH P-256 est générée sur l'appareil ; la clé publique est enregistrée en base ; la clé privée n'est stockée en base qu'**enveloppée** (AES-GCM sous une clé dérivée d'un mot de passe de chiffrement par PBKDF2-SHA256, 600 000 itérations, sel aléatoire [16]) et, sur l'appareil, comme clé **non extractable** (IndexedDB).
- **Clés partagées** : ECDH entre la clé privée locale et la clé publique du partenaire, puis HKDF-SHA256 [15] avec deux informations de contexte distinctes pour obtenir une clé de messages (AES-GCM 256) et une clé d'authentification (HMAC-SHA-256) pour la signalisation d'appel.
- **Messages** : AES-GCM avec IV aléatoire de 96 bits ; les données authentifiées additionnelles lient le chiffré au couple, à l'auteur et au type de message (un chiffré ne peut pas être déplacé vers un autre auteur ou type). Format « 1.<base64(IV|chiffré)> ».
- **Médias** : une clé AES-GCM aléatoire par fichier ; le fichier chiffré est téléversé (type octet-stream) et la clé voyage dans un message chiffré.
- **Appels** : le MAC HMAC porte sur les empreintes DTLS de l'offre ou de la réponse, l'identifiant d'appel et le rôle ; un appel dont le MAC est invalide est refusé ; un appel est refusé au départ si le chiffrement est requis mais verrouillé.

- **Vérification** : un code de sécurité de 12 groupes de 5 chiffres, identique des deux côtés si les clés correspondent, est affiché ; une empreinte de la clé du partenaire est mémorisée à la première vue (confiance à la première utilisation, TOFU) et un changement bloque l'envoi jusqu'à confirmation.
- **Non-régression** : dès que les deux membres ont une clé, un déclencheur SQL refuse les messages en clair ; un message ne peut pas voir son marqueur « enc » modifié.

Limites assumées. Pas de secret de transmission aval (clés statiques : le vol d'une clé privée ouvre les messages passés) ; métadonnées (qui, quand, type, taille, durée des vocaux, identifiant d'appel) non masquées ; perte irrécupérable du mot de passe de chiffrement ; fichiers chiffrés en un seul bloc en mémoire (adapté à la limite de 10 Mo, non à de grands fichiers) ; pas d'audit externe ; les anciens messages ne sont pas chiffrés rétroactivement.

4.13. Limites de l'implémentation

- Pas de métriques collectées (ni côté client, ni côté serveur) : la latence, le temps d'établissement et le taux d'échec ne sont pas mesurables à partir de l'application actuelle.
- Limiteur de débit en mémoire par instance ; donc non partagé entre instances serverless.
- La reprise du micro et du Wake Lock dépendent de la prise en charge des navigateurs.
- Appel entrant application fermée : notification seulement, sans sonnerie.
- Une seule conversation par couple, sans groupe.

4.14. Problèmes techniques rencontrés et corrections

Cette section applique le format : symptôme, cause, diagnostic, correction, impact, limites, validation. Aucun problème n'est déclaré résolu sans élément de preuve ; le statut est indiqué explicitement.

P1 — Messages vocaux rejetés par le stockage

Statut : corrigé et vérifié par essai

Symptôme	Les messages vocaux « ne partent pas » (signalé par l'utilisateur).
Cause	Le bucket privé n'autorisait que quatre types MIME d'image (migration 0001) ; tout téléversement audio était refusé.
Diagnostic	Lecture de la configuration du bucket et de l'erreur de téléversement.
Correction	Migration 0009 ajoutant audio/webm, audio/mp4, audio/aac, audio/mpeg, audio/ogg, audio/x-m4a.
Impact	Les enregistrements sont acceptés par le stockage.
Limites	Dépend de l'application de la migration à la base de production.
Validation	Téléversements réels de fichiers audio/webm et audio/mp4 réussis par le développeur ; pas de campagne sur appareils.

P2 — Messages vocaux vides ou lents

Statut : corrections livrées, effet non mesuré

Symptôme	Messages reçus parfois vides ; envoi jugé lent (signalés par l'utilisateur).
Cause	Hypothèses non démontrées : conteneur WebM non lisible par certains Safari ; type MIME de MediaRecorder parfois vide sur Safari ; débit audio par défaut élevé pour de la voix.

Diagnostic	Analyse du code de l'enregistreur et du comportement documenté de MediaRecorder.
Correction	MP4/AAC en premier, débit demandé 32 kbit/s, type de repli, refus d'envoyer un enregistrement vide (< 200 octets ou sans tranche).
Impact	Réduction attendue de la taille des fichiers (non mesurée).
Limites	La cause réelle sur les appareils concernés n'est pas confirmée.
Validation	À faire : protocole 5.3 (S-V : taille, durée d'envoi, lisibilité sur iOS/Android).

P3 — Appel audio sans son distant

Statut : correction livrée, confirmation utilisateur non documentée

Symptôme	Absence de son de l'autre personne en appel audio.
Cause	Le flux distant ne contenant que de l'audio n'était attaché à aucun élément média ; seul l'affichage vidéo en attachait un.
Diagnostic	Lecture du composant d'affichage : l'élément n'existait que pour la vidéo.
Correction	Élément <video> invisible rattaché au flux distant en appel audio ; relance de play() sur les événements de chargement, d'ajout de piste et de retour de visibilité ; suppression d'une analyse audio secondaire sur le flux distant, susceptible de couper le son sous Safari (hypothèse).
Impact	Lecture de l'audio distant.
Limites	Le comportement exact sur Safari iOS reste à vérifier sur appareil.
Validation	À faire : S-A (appel audio bout en bout sur deux appareils).

P4 — Appels coupés quand l'application passe en arrière-plan

Statut : cause supprimée ; limite de plateforme persistante

Symptôme	Perte du micro puis coupure de l'appel après un certain temps ou à la sortie de l'application.
Cause	Un gestionnaire pagehide raccrochait l'appel ; iOS déclenche pagehide lors d'un simple passage en arrière-plan. Aucun Wake Lock n'empêchait l'extinction de l'écran.
Diagnostic	Revue du cycle de vie de la page et des gestionnaires d'événements.
Correction	Suppression du gestionnaire ; Wake Lock d'écran pendant l'appel connecté ; reprise du micro si la piste est terminée au retour au premier plan.
Impact	L'application ne termine plus volontairement l'appel.
Limites	Le système peut suspendre une PWA ; aucune API web ne garantit le micro en arrière-plan sur iOS.
Validation	À faire : scénarios S-5 et S-6 sur appareils.

P5 — Latence d'établissement des appels

Statut : optimisations livrées, effet non mesuré

Symptôme	Établissement jugé lent (signalé par l'utilisateur).
Cause	Analyse du code : récupération des serveurs ICE à la demande sur le chemin critique des deux côtés ; appelé informé par la réplique de la ligne de base plutôt que par la diffusion ; vérifications de nettoyage séquentielles dans l'action de démarrage.
Diagnostic	Lecture du flux d'appel ; aucune mesure chronométrée n'a été effectuée.
Correction	Cache de 2 h et préchargement des serveurs ICE ; réservoir de candidats ; offre diffusée portant le type d'appel ; nettoyage des appels périmés seulement s'il existe un appel actif.
Impact	Réduction attendue (non quantifiée).

Limites	Les causes réelles dominantes (réseau, relais) ne sont pas isolées sans instrumentation.
Validation	À faire : mesure du temps d'établissement avant/après (5.3, métrique M1).

P6 — Bouton haut-parleur sans effet

Statut : corrigé dans le code

Symptôme	Le bouton ne produisait aucun changement audible ni visuel.
Cause	Sélecteur ciblant un élément inexistant ; aucun état visuel ; l'API setSinkId n'existe pas partout.
Diagnostic	Revue du gestionnaire.
Correction	Ciblage de l'élément audible, état affiché cohérent avec la sortie réelle, bouton masqué si l'API ou plusieurs sorties sont absentes.
Impact	Fonctionnel là où setSinkId est disponible.
Limites	Indisponible sur Safari iOS (limite du navigateur, non contournée).
Validation	À faire sur appareil Android/Chrome.

P7 — Notifications push non reçues en production

Statut : correction livrée ; réception en production non documentée

Symptôme	Pas de notification hors de l'application.
Cause	En environnement serverless, une promesse d'envoi non attendue est suspendue dès que la réponse est renvoyée ; l'envoi n'aboutissait pas.
Diagnostic	Analyse du modèle d'exécution de la plateforme.
Correction	Exécution de l'envoi dans after() de Next.js jusqu'à la réponse du service push.
Impact	L'envoi est mené à terme avant la fin de la fonction.
Limites	Dépend des clés VAPID et de la clé de service définies dans l'environnement de production.
Validation	À faire : S-N (réception sur appareils, application fermée).

P8 — Interaction entre le micro de l'appel et l'enregistrement vocal

Statut : mesures préventives ; conflit non reproduit formellement

Symptôme	Soupçon d'enregistrements vocaux déclenchés ou perturbés pendant un appel audio (signalé par l'utilisateur).
Cause	Aucun mécanisme créant automatiquement un message vocal n'a été trouvé. Hypothèse : contention sur la ressource micro (plusieurs getUserMedia).
Diagnostic	Revue des flux : l'enregistreur et l'appel ouvrent des flux distincts ; aucun partage.
Correction	Bouton d'enregistrement désactivé pendant un appel ; enregistrement en cours abandonné si un appel démarre ; aucun envoi d'enregistrement vide.
Impact	Le micro appartient à l'appel durant l'appel.
Limites	La cause observée par l'utilisateur n'a pas été identifiée avec certitude.
Validation	À faire : essai d'appel avec tentative d'enregistrement sur iOS.

P9 — Sonneries et tonalités silencieuses

Statut : correction livrée, confirmation utilisateur non documentée

Symptôme	Aucune sonnerie ni tonalité d'établissement/raccrochage.
Cause	Le code testait l'état « running » du contexte audio immédiatement après resume(), qui est asynchrone ; un contexte suspendu ou interrompu (par exemple à l'ouverture du micro) ne jouait rien. Sur iOS, l'audio web peut aussi être coupé par le commutateur silencieux.
Diagnostic	Revue du module audio.
Correction	Attente effective de la reprise avant lecture ; déblocage par un tampon silencieux au premier geste ; session audio déclarée en lecture pendant la sonnerie puis rétablie.
Impact	Lecture possible après un premier geste.
Limites	Aucun son avant le premier geste de la session ; pas de sonnerie hors application.
Validation	À faire sur appareils.

P10 — Appels qui « ne s'établissent plus »

Statut : NON confirmé comme résolu

Symptôme	Même après décrochage, l'appel ne s'établit pas, en audio comme en vidéo (signalé le 25 septembre).
Cause	Non établie. Hypothèses : relais TURN non configuré en production (appels entre réseaux différents) ; état de la session audio iOS après la sonnerie.
Diagnostic	Aucune reproduction ; revue du code.
Correction	Réinitialisation de la session audio avant la capture du micro ; message explicite « relais non configuré » lorsqu'aucun serveur TURN n'est fourni ; journalisation des erreurs d'appel.
Impact	Meilleure capacité de diagnostic.
Limites	Le problème peut persister ; la cause n'est pas isolée.
Validation	À faire : S-1 et S-6 avec journalisation (type de candidat sélectionné).

P11 — Messages absents au retour d'arrière-plan

Statut : corrigé dans le code

Symptôme	Les nouveaux messages n'apparaissent qu'après avoir quitté puis rouvert la conversation.
Cause	Les événements temps réel émis pendant l'absence ne sont pas rejoués.
Correction	Relecture des messages récents au retour de visibilité et au retour en ligne ; resynchronisation des pastilles de non-lu.
Limites	Limitée aux 100 messages suivants le dernier connu.
Validation	À faire : S-5.

P12 — Débordement de la mise en page avec un message cité long

Statut : corrigé, vérifié visuellement

Symptôme	Le bouton d'envoi et la croix de réponse sortaient de l'écran ; la conversation s'élargissait.
Cause	Colonnes de grille dimensionnées par leur contenu.
Correction	Colonnes bornées (minmax(0, 1fr)), bulles limitées à leur colonne, citation tronquée.
Validation	Rendu de composants avec données factices à 375 px de large (captures d'écran du développeur).

P13 — Chargement des photos

Statut : optimisations livrées, effet non mesuré

Symptôme	Chargement lent des éléments (demande initiale du projet).
Cause	Signature répétée des mêmes chemins ; images volumineuses.
Correction	Cache de signatures au niveau du module et déduplication des requêtes en vol ; signature par lot ; chargement différé des images ; redimensionnement à 1 800 px et WebP à l'envoi.
Validation	À faire : mesure du temps de chargement (métrique M7).

CHAPITRE 5 — EXPÉRIMENTATION ET ÉVALUATION

Ce chapitre distingue sans ambiguïté ce qui a été **mesuré** de ce qui est **protocolé**. Conformément à la règle de non-invention, aucun résultat de latence, de perte, de reprise ou de qualité n'est donné : ces grandeurs ne sont pas collectées par l'application actuelle et aucune campagne n'a été menée. Les seuls résultats expérimentaux sont ceux des tests automatisés et du banc de calcul (section 5.7).

5.1. Méthodologie expérimentale

La méthode suit le paradigme But–Question–Métrique [40] : chaque scénario part d'un but (vérifier une propriété), formule une hypothèse mesurable et définit les métriques et la procédure. Les critères d'acceptation sont fixés **avant** la mesure. Pour chaque condition, on recommande au moins 30 répétitions, on rapporte la médiane, le 95e percentile et l'écart interquartile, et on traite séparément les échecs (taux de réussite) des durées (calculées sur les réussites). Les mesures de durée à l'intérieur d'un même appareil utilisent l'horloge monotone du navigateur (`performance.now()`) afin de ne pas dépendre de la synchronisation des horloges.

5.2. Environnement de test

Tableau 11 — Environnements : réalisé et prévu

Environnement	Statut	Usage
Poste de développement Windows 11, Node.js v26.1.0 (win32/x64)	Utilisé	Tests SQL (PGLite), tests et banc cryptographiques, compilation et lint
Navigateur du poste en émulation de gabarit mobile (375 × 812)	Utilisé	Vérification visuelle de composants avec données factices
Deux téléphones réels (iOS Safari PWA, Android Chrome)	Non utilisé pour des mesures	Protocole S-1 à S-6
Émulation réseau au niveau du système	Non utilisé	Protocole S-2 à S-6
Plateforme de production (Vercel, Supabase, Cloudflare)	Non audité	Configuration à vérifier

5.3. Métriques et instrumentation proposées

Tableau 12 — Métriques

Réf.	Métrique	Définition et source
M1	Temps d'établissement	De l'appui sur « Appeler » à <code>connectionState = connected</code> (horloge locale de l'appelant)
M2	Délai de sonnerie	De l'appui sur « Appeler » à la réception de « ringing » (appellant)
M3	Taux de réussite d'établissement	Appels connectés / appels initiés
M4	Délai aller-retour	<code>currentRoundTripTime</code> de la paire de candidats sélectionnée (<code>getStats()</code>)
M5	Gigue et pertes	<code>jitter</code> et <code>packetsLost</code> des flux entrants (<code>getStats()</code>)
M6	Débit	<code>bytesReceived/bytesSent</code> échantillonnés par seconde
M7	Temps de chargement / d'envoi d'un média	Du déclenchement à l'affichage / à la confirmation de l'action
M8	Temps de reprise	De la perte de connectivité (<code>disconnected</code>) au retour à <code>connected</code>

Réf.	Métrique	Définition et source
M9	Type de chemin	Type de candidat de la paire sélectionnée : host, srflx ou relay
M10	Délai de livraison d'un message	De l'envoi à l'affichage chez le destinataire (nécessite deux horloges synchronisées ou un écho mesuré)
M11	Coût de chiffrement	Temps de calcul des primitives (mesuré, 5.7)

Proposition d'instrumentation (non implémentée). Émettre des événements horodatés (call_start, offer_sent, offer_received, answer_sent, ice_connected, media_flowring, call_end, reconnect_start, reconnect_end ; voice_record_start/end, upload_start/end) vers un journal d'essai local ou une table dédiée sans contenu ; échantillonner getStats() chaque seconde pendant l'appel ; stocker M9 pour distinguer chemin direct et chemin relayé. Cette instrumentation doit préserver la confidentialité (aucun contenu, identifiants pseudonymisés) et être désactivée par défaut.

5.4. Scénarios de test

Les huit scénarios sont présentés selon les rubriques imposées. Pour les scénarios S-1 à S-7, la rubrique « résultats » est « non mesurés à ce jour ».

S-1 — Réseau stable

Environnement	Deux appareils réels (iOS et Android) sur le même Wi-Fi, puis sur deux réseaux différents (Wi-Fi / 4G) ; version de l'application figée.
Hypothèse	Sur réseau stable, l'appel s'établit dans un délai court (valeur cible à fixer a priori, par exemple < 3 s) et le taux de réussite est élevé ; sur réseaux différents, il dépend de la présence du relais TURN (H3).
Procédure	30 appels audio et 30 vidéo par configuration réseau, alternance des rôles ; relevé de M1, M2, M3, M9.
Métriques	M1 temps d'établissement ; M2 délai de sonnerie chez l'appelé ; M3 taux de réussite ; M9 type de candidat (host/srflx/relay).
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet tant qu'aucune mesure n'est disponible.
Limites	Deux appareils seulement ; échantillon de petite taille ; opérateurs et NAT non contrôlés.

S-2 — Latence élevée

Environnement	Appel sur un même réseau, dégradation introduite au niveau du système (netem sous Linux, Clumsy sous Windows, Network Link Conditioner d'Apple). <i>La limitation réseau des outils de développement du navigateur ne s'applique pas au trafic WebRTC en UDP.</i>
Hypothèse	L'augmentation du délai aller-retour dégrade la qualité perçue au-delà d'environ 150 ms unidirectionnel [24] et allonge l'établissement proportionnellement au nombre d'échanges de signalisation.
Procédure	Paliers de délai ajouté : 0, 50, 100, 200, 400 ms ; 10 appels par palier ; relevé de M1, M4, M5.
Métriques	M1, M4 (RTT du chemin sélectionné), M5 (gigue, pertes).
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet.
Limites	Le délai de signalisation (WebSocket) et le délai média sont affectés ensemble si la dégradation est globale ; pour les séparer, appliquer le filtre aux seuls ports média.

S-3 — Perte de paquets

Environnement	Idem S-2, avec perte aléatoire.
Hypothèse	Une perte de 1 à 5 % reste tolérable pour la voix (dissimulation de perte) mais dégrade rapidement la vidéo.
Procédure	Paliers 0, 1, 2, 5, 10 % ; 10 appels par palier ; échantillonnage de getStats() chaque seconde.
Métriques	M5 (paquets perdus, gigue), débit reçu, évaluation subjective de la qualité par échelle d'opinion à 5 niveaux.
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet.
Limites	Évaluation subjective à encadrer (ordre aléatoire, aveugle au palier) ; tests sur une seule paire d'appareils.

S-4 — Bande passante limitée

Environnement	Limitation de débit à 2 Mbit/s, 500, 250, 100 kbit/s.
Hypothèse	L'adaptation de débit des navigateurs maintient l'audio mais réduit la résolution ou la fluidité vidéo ; le chargement des photos et des vocaux s'allonge proportionnellement à leur taille.
Procédure	Appels vidéo de 2 minutes par palier ; envoi d'une photo de 3 Mo et d'un vocal de 30 s.
Métriques	Débit émis et reçu, résolution reçue, M7 temps de chargement/envoi des médias.
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet.
Limites	Les algorithmes d'adaptation dépendent de la version du navigateur.

S-5 — Coupure temporaire

Environnement	Coupure du réseau d'un appareil pendant 5, 15, 25 s (mode avion ou coupure du Wi-Fi), pendant un appel connecté et pendant une conversation ouverte.
Hypothèse	Pour une coupure < 20 s, l'appel reprend ; au-delà, il se termine en « failed ». La conversation rattrape les messages manquants au retour en ligne.
Procédure	10 essais par durée ; relevé du temps de reprise (M8), du statut final de l'appel et du nombre de messages rattrapés.
Métriques	M8 temps de reprise ; taux de reprise ; messages manquants après rattrapage.
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet.
Limites	Le comportement iOS en arrière-plan ou écran verrouillé peut dominer le résultat (hors contrôle de l'application).

S-6 — Changement de réseau

Environnement	Appel connecté sur Wi-Fi, bascule vers la 4G (désactivation du Wi-Fi), et inversement.
Hypothèse	Le redémarrage d'ICE par l'appelant rétablit le média dans la limite de 3 tentatives et de 20 s ; la reprise est plus lente si l'appelé change de réseau.
Procédure	10 essais par sens et par rôle (appelant / appelé change de réseau).
Métriques	M8, M9 (nouveau chemin), taux de réussite.
Observations / résultats	Non mesurés à ce jour.
Interprétation	Sans objet.

Limites	Dépend de la configuration TURN en production et du fournisseur d'accès.
---------	--

S-7 — Charge croissante

Environnement	Non réalisable de façon pertinente : la plateforme sert deux utilisateurs et n'a pas de mode de simulation de charge ; les limites des offres gratuites des fournisseurs ne doivent pas être sollicitées.
Hypothèse	—
Procédure	Perspective : tests de charge de l'API de messagerie (débit de messages par seconde) sur un environnement dédié, avec comptes de test.
Métriques	Débit, latence p95 des actions, taux d'erreur.
Observations / résultats	Non applicable.
Interprétation	—
Limites	Les conclusions sur la scalabilité ne peuvent pas être tirées du prototype actuel.

S-8 — Tests de sécurité

Environnement	Base PostgreSQL en mémoire (PGLite) jouant les 10 migrations avec des simulations d'authentification ; poste de développement ; Node.js v26.1.0.
Hypothèse	Les politiques RLS, déclencheurs et contraintes empêchent l'accès croisé, l'usurpation d'auteur, la falsification des champs immuables et la lecture des secrets ; les primitives cryptographiques détectent altération et déplacement de chiffrés.
Procédure	Exécution de <code>npm run test:db</code> et <code>npm run test:e2e</code> ; exécution du banc <code>scripts/e2e-bench.mjs</code> .
Métriques	Nombre de contrôles réussis ; temps de calcul (médiane, p95) ; surcoût de taille.
Observations / résultats	Mesurés : 96 contrôles de base réussis (0 échec) ; 22 contrôles cryptographiques réussis ; mesures de calcul en 5.7.
Interprétation	Les garanties d'isolation et d'intégrité testées sont vérifiées dans un environnement simulé ; voir limites.
Limites	Environnement simulé et non la plateforme Supabase réelle ; pas de test d'intrusion externe ; pas de test du transport (TLS), de la signalisation sous attaque active ni des politiques du Storage dans leur service réel.

5.5. Évaluation de la sécurité

Mesuré (S-8). La suite de tests de base rejoue les migrations et vérifie notamment : l'absence de lecture croisée entre couples (un tiers ne voit aucun message, aucun appel, aucune clé), l'impossibilité d'usurper l'auteur d'un message ou l'appelant d'un appel, l'immutabilité des champs fixes (auteur, type, chemin de fichier, durée, marqueur de chiffrage), l'inaccessibilité des tables du verrou d'application (même pour leur propriétaire), la confidentialité de la clé privée enveloppée vis-à-vis du partenaire, le refus des messages en clair lorsque les deux membres ont une clé et le rejet d'un nombre d'itérations de dérivation trop faible. 96 contrôles sont réussis, aucun n'échoue. La suite cryptographique vérifie qu'un tiers ne déchiffre pas, qu'un chiffré déplacé vers un autre auteur ou type est rejeté, qu'une altération est détectée, que la sauvegarde de clé exige le bon mot de passe, que le code de sécurité est identique des deux côtés et que le MAC de signalisation détecte une empreinte substituée, un changement de rôle, d'appel ou de signataire. 22 contrôles sont réussis.

Non mesuré / non testé. TLS et configuration des hébergeurs ; comportement du service Storage réel ; interposition active dans la signalisation en conditions réelles ; contournement du limiteur de débit en environnement multi-instances ; révocation de session ; fuite de métadonnées dans les journaux de la plateforme ; test d'intrusion.

5.6. Évaluation de la résilience

La résilience est ici évaluée **uniquement par revue du mécanisme** (3.9 et 3.11). L'efficacité de la reprise (S-5, S-6) n'est pas démontrée. Les temps de 20 s, 45 s, 60 s et le nombre de 3 redémarrages d'ICE sont des paramètres de conception dont la pertinence doit être éprouvée par les scénarios.

5.7. Résultats obtenus

Les mesures de calcul ci-dessous ont été obtenues avec le script `scripts/e2ee-bench.mjs` (WebCrypto de Node.js v26.1.0, Windows x64) : 300 répétitions pour les textes, 30 pour les fichiers, 100 pour la dérivation ECDH+HKDF, 30 pour la génération de clé, 8 pour la déprotection de la clé (PBKDF2), après échauffement. Ce sont des **coûts de calcul sur un poste de bureau**, pas des temps de bout en bout, et pas des mesures sur téléphone.

Tableau 13 — Coût de calcul de l'E2EE (ms, médiane / 95e percentile)

Opération	Médiane	p95	Remarque
Génération de la paire de clés ECDH P-256	6,58	30,0	Une fois par activation
Dérivation ECDH + HKDF (clés partagées)	2,83	77,0	Une fois par session
Chiffrement d'un texte de 20 caractères	0,23	1,40	AES-GCM
Chiffrement d'un texte de 200 caractères	0,26	1,38	
Chiffrement d'un texte de 1 000 caractères	0,50	4,44	
Chiffrement d'un texte de 4 000 caractères	1,13	6,21	
Déchiffrement d'un texte de 4 000 caractères	0,34	1,79	
Chiffrement d'un fichier de 50 ko	0,61	9,26	AES-GCM, clé par fichier
Chiffrement d'un fichier de 300 ko	2,08	7,05	
Chiffrement d'un fichier de 3 Mo	19,1	38,6	
Déchiffrement d'un fichier de 3 Mo	18,5	34,3	
Signature / vérification du MAC d'un SDP	0,17 / 0,17	0,89 / 0,68	Par offre et par réponse
Calcul du code de sécurité	0,11	0,57	
Déverrouillage de la clé (PBKDF2, 600 000 itérations)	1 947	2 166	Coût volontaire, à l'activation/restauration

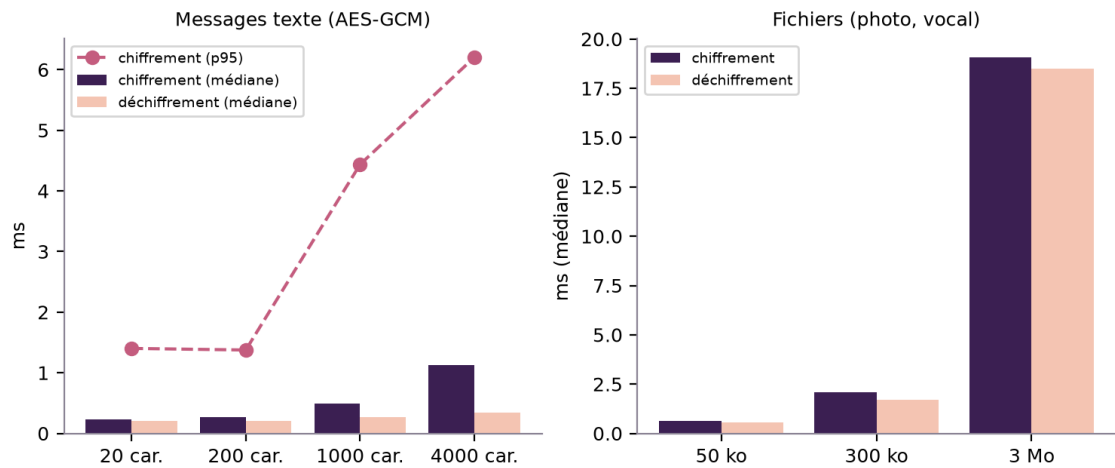


Figure 11 — Coût de calcul du chiffrement (mesures réelles, médiane et p95)

Tableau 14 — Surcoût de taille du chiffrement

Objet	Taille claire	Taille chiffrée	Surcoût
Texte	20 car.	66 car.	× 3,3
Texte	200 car.	306 car.	× 1,53
Texte	1 000 car.	1 374 car.	× 1,37
Texte	4 000 car.	5 374 car.	× 1,34
Fichier	toute taille	+ 28 octets	12 (IV) + 16 (étiquette)

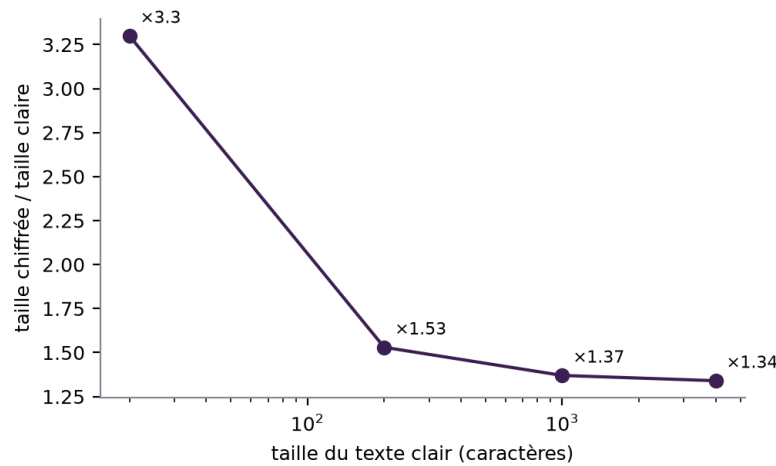


Figure 12 — Surcoût de taille d'un texte chiffré (base64, IV et étiquette inclus)

Tableau 15 — Autres vérifications automatiques

Vérification	Résultat
Tests d'isolation SQL (npm run test:db)	96 contrôles réussis, 0 échec
Tests cryptographiques (npm run test:e2ee)	22 contrôles réussis
Compilation de production (next build)	Réussie
Analyse statique (ESLint)	0 erreur, 2 avertissements (attribut alt d'un composant de logo)

5.8. Analyse des résultats

- **Analyse 1.** Sur le poste de test, le calcul de chiffrement d'un message courant est inférieur à 1,2 ms (médiane) et celui d'une photo de 300 ko de l'ordre de 2 ms. Rapporté à un budget de 150 ms de délai unidirectionnel [24], ce coût est faible. **Réserve :** l'hypothèse H2 n'est que partiellement testée, car le coût sur un téléphone d'entrée de gamme, l'occupation mémoire et l'effet sur la latence de bout en bout ne sont pas mesurés.
- **Analyse 2.** Le coût dominant du chiffrement est d'**usage**, non de calcul : la gestion d'un mot de passe de chiffrement, le déverrouillage d'environ 2 s sur le poste de test (volontairement coûteux), la perte d'accès en cas d'oubli. Le compromis se déplace de la performance vers l'ergonomie et la disponibilité des données.
- **Analyse 3.** Le surcoût de taille des textes est amorti au-delà de quelques centaines de caractères ($\times 1,34$ limite due au codage base64) ; il est négligeable pour les fichiers (28 octets).
- **Analyse 4.** Les 96 contrôles montrent que les garanties d'isolation sont exprimées dans la base et non dans l'interface. Ils ne démontrent pas l'absence de vulnérabilité : le périmètre est celui des cas testés.
- **Constat d'absence de preuve.** Les propriétés de latence d'établissement, de gigue, de perte, de reprise et de taux de réussite, centrales dans la problématique, ne peuvent pas être évaluées sans l'instrumentation et la campagne décrites en 5.3 et 5.4.

CHAPITRE 6 — DISCUSSION, LIMITES ET PERSPECTIVES

6.1. Discussion des résultats

Les résultats obtenus valident la **faisabilité** d'une architecture qui combine contrôle d'accès au niveau de la base, signalisation sans serveur dédié, média pair-à-pair avec relais éphémère et chiffrement de bout en bout optionnel à faible coût de calcul. Ils ne valident pas les propriétés de performance et de disponibilité en conditions réelles. La contribution principale de l'étude est donc double : une description vérifiable de l'architecture, et un cadre expérimental précis pour la compléter.

6.2. Adéquation entre les objectifs et les résultats

Tableau 16 — Objectifs et degré d'atteinte

Objectif	Degré d'atteinte	Justification
Reconstituer l'architecture réelle	Atteint	Chapitres 2–4, matrice en annexe A
Analyser la signalisation et la reconnexion	Atteint (analyse), non mesuré	3.4, 3.5, 3.9
Étudier médias et persistance	Atteint	3.6, 3.7, 4.4, 4.5
Analyser le modèle de sécurité et ses limites	Atteint	3.10, 4.12, 5.5
Documenter les problèmes rencontrés	Atteint, plusieurs sans validation	4.14
Évaluer expérimentalement performance et résilience	Partiel	Calcul et sécurité mesurés ; réseau protocolé seulement

6.3. Compromis entre sécurité, performance et disponibilité

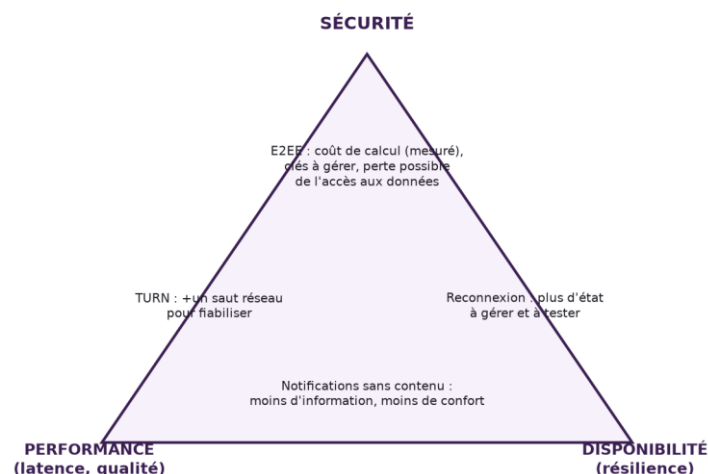


Figure 13 — Triangle des compromis sécurité – performance – disponibilité

Tableau 17 — Analyse des compromis

Mécanisme	Gain de sécurité	Coût en performance	Coût en disponibilité / usage	Preuve
E2EE applicatif	Contenu illisible par l'opérateur	Calcul faible (mesuré) ; taille $\times 1,34$	Mot de passe à gérer ; perte possible des données ; restauration sur autre appareil	Mesure de calcul ; reste non mesuré
MAC des empreintes DTLS	Résiste à une interposition dans la signalisation	< 1 ms par offre/réponse (mesuré)	Refus d'appel si clé verrouillée ou changée	Mesure + tests unitaires
Relais TURN	Neutre (le média reste chiffré)	Saut réseau supplémentaire (non mesuré)	Améliore la connectivité entre réseaux différents (hypothèse)	Aucune mesure
RLS + validation serveur	Isolation par couple	Évaluation des politiques à chaque requête (non mesuré)	Complexité des migrations	96 contrôles
Notifications sans contenu	Pas de fuite via le service push	Neutre	Notification moins informative	Revue du code
Verrou d'application	Protège un appareil déverrouillé prêt	Neutre	Étape supplémentaire à chaque ouverture	Revue du code
Reprise automatique d'appel	—	Redémarrage d'ICE : charge de signalisation	Plus d'états à gérer	Aucune mesure

Analyse. Aucune solution n'est « meilleure » dans l'absolu : activer l'E2EE protège contre un opérateur curieux mais aggrave le risque de perte de données ; maintenir la reconnexion améliore la continuité mais complexifie l'état ; minimiser les métadonnées réduit les possibilités de diagnostic, donc l'observabilité. Le triangle est un espace d'arbitrage : l'architecture cible doit expliciter ses choix, ses hypothèses de menace et ses indicateurs.

6.4. Apports du prototype

- Un cas concret de séparation entre état de communication et affichage (hypothèse H1), documenté et implémenté.
- Une signalisation double (diffusion + base) répondant au caractère non fiable de la diffusion.
- Une couche E2EE optionnelle de bout en bout (messages, fichiers, signalisation) avec refus de rétrogradation côté base et vérification par code de sécurité.
- Un recueil de problèmes réels et de corrections, avec statut de preuve.
- Un protocole expérimental prêt à l'emploi.

6.5. Limites actuelles

6.5.1. Limites liées à l'environnement PWA

- **CallKit/PushKit absents.** Une PWA ne peut pas présenter l'interface d'appel native du système ni être réveillée par un push VoIP ; l'appel entrant application fermée se réduit à une notification.
- **Arrière-plan.** Le système d'exploitation peut suspendre la page, interrompre la capture du micro ou verrouiller l'audio ; l'application peut réduire ce risque (Wake Lock, reprise du micro) mais pas l'empêcher.
- **Audio web.** Verrouillé avant un premier geste ; sensible au commutateur silencieux sur iOS ; setSinkId indisponible sur Safari iOS.

- **Notifications.** Dépendent de l'installation de la PWA, de la permission et des politiques de la plateforme ; supprimer une notification peut, selon la plateforme, être sanctionné par le retrait de l'autorisation (risque signalé, non quantifié).

6.5.2. Limites liées aux communications temps réel

- Sans relais TURN configuré, deux téléphones sur des réseaux différents peuvent ne pas se joindre.
- Pas d'adaptation volontaire du débit ni des codecs par l'application : elle s'en remet au navigateur.
- Appels uniquement un-à-un, en pair-à-pair ; pas d'appel de groupe.
- Aucune métrique de qualité collectée.

6.5.3. Limites méthodologiques

- Peu d'appareils, pas de campagne ; résultats de calcul sur un poste de bureau ; configuration de production non vérifiable depuis le dépôt ; bibliographie à revérifier.

6.6. Perspectives d'amélioration

Les éléments suivants sont des propositions, non confirmées comme implémentées.

- **Observabilité** : instrumentation décrite en 5.3, export des événements d'appel et des statistiques WebRTC, tableau de bord de taux de réussite et de temps d'établissement [33][38].
- **E2EE renforcé** : protocole à cliquet (type Double Ratchet) apportant le secret de transmission aval et la récupération après compromission [30][29] ; rotation des clés ; vérification des clés par un tiers de confiance ; audit externe.
- **Résilience** : relais TURN redondants multi-régions ; reprise de téléversement ; file d'envoi hors ligne ; mesure et réglage des délais (20 s, 45 s, 60 s) d'après les scénarios S-5 et S-6.
- **Cloud et scalabilité** : limitation de débit distribuée ; séparation en services ; SFU pour des appels de groupe ; tests de charge sur environnement dédié.
- **Migration native éventuelle** : application iOS native avec **CallKit** (interface d'appel système) et **PushKit** (push VoIP) et notifications natives ; cette perspective n'est pas partie de l'implémentation actuelle.
- **Fichiers génériques** : transfert chiffré de documents, avec limites de taille et analyse antivirus.
- **Métadonnées** : réduire l'information journalisée, étudier l'acheminement par relais pour masquer les adresses IP.

6.7. Perspectives de déploiement à plus grande échelle

Le passage à plus de deux utilisateurs impose de revoir les curseurs de lecture (un par membre), l'indicateur de saisie, la signalisation d'appel (appels de groupe via SFU), la gestion des clés (chiffrement de groupe) et la limitation de débit. Ces évolutions modifient la nature du problème ; elles ne sont pas déductibles des résultats du prototype.

CONCLUSION GÉNÉRALE

Rappel du problème. Ce mémoire cherchait à savoir comment concevoir une architecture garantissant simultanément confidentialité, intégrité, disponibilité et faible latence dans une plateforme de communication multimédia temps réel fonctionnant sur des réseaux hétérogènes.

Démarche. À partir d'un audit du dépôt du prototype Allyza, l'architecture réelle du module de messagerie a été reconstituée, classée par niveau de preuve, confrontée à l'état de l'art et complétée par des mesures de calcul et des tests automatisés. Les problèmes réellement rencontrés ont été consignés avec leur statut de validation.

Réponse, dans les limites des preuves. (i) *Confidentialité* : vis-à-vis des tiers, elle repose sur l'authentification, les politiques RLS, le stockage privé et le transport chiffré ; vis-à-vis de l'opérateur, elle n'est obtenue que lorsque le chiffrement de bout en bout optionnel est activé par les deux personnes, mécanisme implémenté et testé unitairement mais non validé de bout en bout (classe B). (ii) *Intégrité* : assurée en base par des contraintes et déclencheurs vérifiés (96 contrôles) et, pour le contenu chiffré et la signalisation, par un chiffrement authentifié et un MAC vérifiés par 22 contrôles. (iii) *Disponibilité* : des mécanismes de reprise existent (double signalisation, rattrapage, redémarrage d'ICE, reprise du micro), mais leur efficacité n'est pas mesurée. (iv) *Faible latence* : le coût de calcul du chiffrement est faible sur le poste de test (inférieur à 1,2 ms pour un message de 4 000 caractères), mais la latence d'établissement et de transmission n'est pas mesurée. (v) *Réseaux hétérogènes* : la nécessité d'un relais TURN est établie par la littérature ; sa présence et son efficacité en production ne sont pas vérifiées.

Conclusion. L'étude ne démontre donc pas que l'architecture garantit simultanément les quatre propriétés. Elle démontre qu'une telle architecture peut être construite avec des composants gérés et des primitives standard, que ses compromis sont explicites et que les propriétés restant à établir sont mesurables par un protocole précis. L'architecture cible et la migration native (CallKit/PushKit) relèvent exclusivement des perspectives.

Ouverture. La suite logique consiste à instrumenter l'application, à conduire les scénarios S-1 à S-6 sur appareils réels avec dégradations réseau contrôlées, puis à réviser les paramètres de reprise et la configuration du relais d'après ces mesures.

BIBLIOGRAPHIE

- [1] Alvestrand, H. — *RFC 8825 : Overview: Real-Time Protocols for Browser-Based Applications*. IETF, 2021.
- [2] Uberti, J., Jennings, C., Rescorla, E. (éd.) — *RFC 8829 : JavaScript Session Establishment Protocol (JSEP)*. IETF, 2021.
- [3] Keränen, A., Holmberg, C., Rosenberg, J. — *RFC 8445 : Interactive Connectivity Establishment (ICE)*. IETF, 2018.
- [4] Petit-Huguenin, M. et al. — *RFC 8489 : Session Traversal Utilities for NAT (STUN)*. IETF, 2020.
- [5] Reddy, T. et al. — *RFC 8656 : Traversal Using Relays around NAT (TURN)*. IETF, 2020.
- [6] Begen, A., Kyzivat, P., Perkins, C., Handley, M. — *RFC 8866 : SDP: Session Description Protocol*. IETF, 2021.
- [7] Rescorla, E. — *RFC 8827 : WebRTC Security Architecture*. IETF, 2021.
- [8] Rescorla, E. — *RFC 8826 : Security Considerations for WebRTC*. IETF, 2021.
- [9] Rescorla, E., Modadugu, N. — *RFC 6347 : DTLS Version 1.2 (2012)* ; McGrew, D., Rescorla, E. — *RFC 5764 : DTLS Extension to Establish Keys for SRTP (2010)*. IETF.
- [10] Baugher, M. et al. — *RFC 3711 : The Secure Real-time Transport Protocol (SRTP)*. IETF, 2004.
- [11] Schulzrinne, H. et al. — *RFC 3550 : RTP: A Transport Protocol for Real-Time Applications*. IETF, 2003.
- [12] Rescorla, E. — *RFC 8446 : The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018.
- [13] Fette, I., Melnikov, A. — *RFC 6455 : The WebSocket Protocol*. IETF, 2011.
- [14] Thomson, M. et al. — *RFC 8030 : Generic Event Delivery Using HTTP Push (2016)* ; *RFC 8291 : Message Encryption for Web Push (2017)* ; *RFC 8292 : VAPID for Web Push (2017)*. IETF.
- [15] Krawczyk, H., Eronen, P. — *RFC 5869 : HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. IETF, 2010.
- [16] Moriarty, K. et al. — *RFC 8018 : PKCS #5: Password-Based Cryptography Specification v2.1*. IETF, 2017.
- [17] Percival, C., Josefsson, S. — *RFC 7914 : The scrypt Password-Based Key Derivation Function*. IETF, 2016.
- [18] Dworkin, M. — *NIST SP 800-38D : Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST, 2007.
- [19] Barker, E. et al. — *NIST SP 800-56A Rev. 3 : Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. NIST, 2018.
- [20] W3C — *WebRTC: Real-Time Communication in Browsers (Recommendation)*. W3C, 2023.
- [21] W3C — *Media Capture and Streams*.
- [22] W3C — *Web Cryptography API (Recommendation)*. W3C, 2017.
- [23] W3C — *Push API ; Service Workers ; Screen Wake Lock API ; Web Authentication ; Web Audio API ; MediaStream Recording ; Audio Output Devices API*.
- [24] ITU-T — *Recommendation G.114 : One-way transmission time*. UIT, 2003.
- [25] Saltzer, J. H., Schroeder, M. D. — « The protection of information in computer systems ». *Proceedings of the IEEE*, 63(9), 1975.

- [26] Brewer, E. — « Towards robust distributed systems ». Keynote, ACM PODC, 2000 ; Gilbert, S., Lynch, N. — « Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services ». *ACM SIGACT News*, 33(2), 2002.
- [27] Avižienis, A., Laprie, J.-C., Randell, B., Landwehr, C. — « Basic concepts and taxonomy of dependable and secure computing ». *IEEE Transactions on Dependable and Secure Computing*, 1(1), 2004.
- [28] Unger, N. et al. — « SoK: Secure Messaging ». *IEEE Symposium on Security and Privacy*, 2015.
- [29] Cohn-Gordon, K., Cremers, C., Dowling, B., Garratt, L., Stebila, D. — « A formal security analysis of the Signal messaging protocol ». *IEEE European Symposium on Security and Privacy*, 2017.
- [30] Perrin, T., Marlinspike, M. — *The Double Ratchet Algorithm*. Signal, spécification, 2016.
- [31] Jennings, C., Hardie, T., Westerlund, M. — « Real-time communications for the web ». *IEEE Communications Magazine*, 51(4), 2013.
- [32] Kurose, J., Ross, K. — *Computer Networking: A Top-Down Approach*. Pearson ; Tanenbaum, A., van Steen, M. — *Distributed Systems*. ; Perkins, C. — *RTP: Audio and Video for the Internet*. Addison-Wesley, 2003.
- [33] Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (éd.) — *Site Reliability Engineering*. O'Reilly, 2016.
- [34] OWASP — *Application Security Verification Standard (v4.0.3, 2021)* ; *Password Storage Cheat Sheet*.
- [35] Documentation officielle : Supabase (Auth, Row Level Security, Realtime, Storage) ; Next.js (App Router, Server Actions, after) ; Cloudflare Realtime TURN ; bibliothèque web-push.
- [36] Audet, F., Jennings, C. — *RFC 4787 : Network Address Translation (NAT) Behavioral Requirements for Unicast UDP*. IETF, 2007.
- [37] Jones, M., Bradley, J., Sakimura, N. — *RFC 7519 : JSON Web Token (JWT)*. IETF, 2015.
- [38] Sridharan, C. — *Distributed Systems Observability*. O'Reilly, 2018.
- [39] Kleppmann, M. — *Designing Data-Intensive Applications*. O'Reilly, 2017.
- [40] Basili, V. R., Caldiera, G., Rombach, H. D. — « The Goal Question Metric Approach ». *Encyclopedia of Software Engineering*, Wiley, 1994.

Remarque de vérification. ces références (RFC, recommandations W3C et ITU-T, publications) ont été citées à partir de sources normatives et académiques connues, sans consultation en ligne durant la rédaction. Les numéros de page, éditions exactes et URL doivent être revérifiés avant dépôt. Les documentations officielles des produits (Supabase, Next.js, Cloudflare) évoluent : consulter la version en vigueur à la date de dépôt.

ANNEXES

Annexe A — Matrice de vérification des fonctionnalités

Classes : **A** réellement implémentée ; **B** partielle ou non validée de bout en bout ; **C** prévue non finalisée ; **D** proposition / perspective. « P » : fourni par la plateforme.

Tableau 18 — Matrice de vérification : élément, classe, preuve dans le projet, technologie, limites

Élément	Cl.	Preuve dans le projet	Technologie	Limites
Messages texte (envoi/réception)	A	messages.ts ; ChatClient.tsx ; migration 0003	Server Actions, PostgreSQL, Realtime	Une seule conversation par couple
Réponse à un message / copie	A	ChatClient.tsx (reply_to)	—	—
Édition (texte)	A	editMessageAction ; trigger messages_guard	SQL	Auteur seulement ; texte seulement
Suppression (logique)	A	deleteMessageAction ; deleted_at ; suppression du fichier	SQL, Storage	Aucune politique DELETE : irréversible côté interface
Stickers (20)	A	stickers.ts ; migration 0005 ; Sticker.tsx	SVG	Jeu fixe défini dans le code
Émojis (sélecteur, messages 1–3 émojis en grand)	A	EmojiPicker.tsx	—	—
Réactions (émoji ou sticker)	A	use-reactions.ts ; toggleReactionAction ; ReactionBar	SQL, Realtime	Sticker codé « s:n » (liste append-only)
Appui long / menu contextuel	A	MessageMenu.tsx	Pointer Events	—
État de lecture (curseur, coche double)	A	message_cursors ; markMessagesReadAction	SQL, Realtime	Pas d'état « livré » distinct
État de livraison distinct	D	—	—	Non implémenté
Indicateur « écrit... »	A	Canal de diffusion couple:<id>	Realtime broadcast	Éphémère
Rattrapage après absence	A	ChatClient.tsx (visibilitychange, online)	—	100 messages au plus
Profils / surnoms	A	nickname.ts ; getPartner()	SQL	Résolveur central
Recherche dans la conversation	A	ChatClient.tsx ; search-store.ts	—	Messages chargés seulement
Liens cliquables	A	Linkified.tsx	—	http(s) seulement
Brouillon local	A	ChatClient.tsx (localStorage)	Web Storage	Non chiffré
Photos (envoi, regroupement, visionneuse, enregistrement)	A	image.ts ; PhotoViewer.tsx	Canvas, Storage, URL signées	Limite 10 Mo ; 6 par envoi
Prise de photo directe	B	input type=file accept=image/*	Navigateur	Dépend du système ; information à vérifier
Documents et fichiers génériques	D	—	—	Bucket limité aux images et audio
Messages vocaux (enregistrement, lecture, forme d'onde)	A	use-voice-recorder.ts ; AudioPlayer	MediaRecorder, Web Audio	Format variable selon navigateur
Appels audio WebRTC	A	CallProvider.tsx ; calls.ts	WebRTC, signalisation Realtime	Fiabilité réseau non mesurée
Appels vidéo WebRTC	A	CallProvider.tsx ; CallOverlay.tsx	WebRTC	Idem
Signalisation d'appel	A	Canal call:<id> ; table calls	Realtime broadcast + postgres_changes	—

Élément	CL	Preuve dans le projet	Technologie	Limites
Relais TURN Cloudflare	A (code) / à vérifier (production)	getIceServersAction	Cloudflare Realtime TURN (REST)	Variables de production non vérifiables
Repli coturn / STUN public	A (code)	calls.ts	coturn, STUN	Non utilisé si Cloudflare configuré
Mute / caméra / inversion caméra	A	CallProvider.tsx	MediaStreamTrack	Inversion non testée sur appareils
Haut-parleur (sortie audio)	B	cycleOutput ; setSinkId	Audio Output Devices API	Indisponible sur Safari iOS
Sonnerie / tonalités	A (code) / B (confirmation)	use-ring.ts ; sfx.ts	Web Audio API	Verrou audio avant le premier geste ; confirmation utilisateur non documentée
Appel entrant application ouverte	A	CallProvider.tsx	Realtime	—
Appel entrant application fermée	B	push.ts ; sw.js	Web Push	Notification seulement, pas de sonnerie
Persistance de l'appel hors écran (palette, retour)	A	CallOverlay.tsx (MiniCall) ; CallProvider	React, état au niveau du layout	—
Persistance en arrière-plan / écran verrouillé	B	Wake Lock ; reprise du micro	Wake Lock API	Décision du système (iOS)
Reconnexion d'appel (ICE restart)	A (code) / non mesuré	onconnectionstatechange	ICE	Efficacité non démontrée
Historique d'appels, rappeler / annuler	A	CallHistory.tsx	SQL	—
Notifications push (messages, appels, manqués)	A (code) / B (production)	push.ts ; sw.js ; after()	Web Push, VAPID	Clés de production non vérifiables
Pastilles « nouveau » persistantes	A	AppShell.tsx ; notifications	SQL, Realtime	—
Authentification	A	auth.ts ; proxy.ts	Supabase Auth, JWT	Révocation instantanée non démontrée
Autorisation (RLS)	A	Migrations 0001–0010	PostgreSQL RLS	Testée en environnement simulé
Verrou d'application (PIN, mot de passe, biométrie)	A	app-lock.ts ; applock.ts ; webauthn-verify.ts	scrypt, WebAuthn	Hors périmètre de la messagerie, sert d'accès
Chiffrement en transit (TLS/WSS)	A (plateforme)	Hébergeurs	TLS	Configuration non audité
Chiffrement de bout en bout	B	crypto.ts ; E2eeProvider ; migration 0010	WebCrypto ECDH/HKDF/AES-GCM	Non validé entre comptes ; production non confirmée ; pas de secret aval
Authentification de la signalisation d'appel (MAC)	B	crypto.ts (signSdp) ; CallProvider	HMAC-SHA-256	Actif seulement avec l'E2EE
Observabilité / métriques	D	—	—	Seulement console.error/warn
CallKit / PushKit / application native	D	—	—	Perspective
Architecture cible	D	—	—	Proposition

Annexe B — Variables d'environnement (noms uniquement)

Les valeurs ne figurent pas dans ce document. L'état de leur définition en production n'est pas vérifiable depuis le dépôt.

Tableau 19 — Variables d'environnement utilisées par la messagerie

Variable	Usage	Portée
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY	Client Supabase (public)	Navigateur + serveur
SUPABASE_SERVICE_ROLE_KEY	Lecture des abonnements push, nettoyage ; jamais exposée	Serveur uniquement
NEXT_PUBLIC_VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_SUBJECT	Web Push	Public / serveur
CLOUDFLARE_TURN_KEY_ID, CLOUDFLARE_TURN_API_TOKEN	Génération des identifiants TURN éphémères	Serveur uniquement
TURN_URLS, TURN_SHARED_SECRET	Alternative coturn	Serveur uniquement
APP_LOCK_SECRET	Signature du cookie de déverrouillage (repli : dérivé de la clé de service)	Serveur uniquement
NEXT_PUBLIC_SITE_URL	Liens de redirection	Public

Annexe C — Événements d'instrumentation proposés

Proposition, non implémentée.

Tableau 20 — Événements proposés pour l'observabilité des appels et des médias

Événement	Contenu	Métriques
call_start / call_end	Début et fin d'appel, motif de fin	M1, M3
offer_sent / offer_received / answer_sent / answer_received	Jalons de la signalisation	M1, M2
ice_state	iceConnectionState et connectionState successifs	M1, M8
selected_pair	Type de candidat (host/srflx/relay), protocole	M9
stats_tick (1 Hz)	RTT, gigue, pertes, débit	M4–M6
reconnect_start / reconnect_end	Redémarrages d'ICE	M8
voice_record_start / end ; upload_start / end	Durée, taille, type MIME, durée d'envoi	M7
media_load	Durée de chargement d'une image ou d'un vocal	M7
message_sent / message_displayed	Chronométrage de l'envoi et de l'affichage	M10

Annexe D — Commandes de reproduction des mesures

- `npm run test:db` : rejoue les 10 migrations dans PGLite et exécute les contrôles d'isolation (96 contrôles).
- `npm run test:e2ee` : tests des primitives cryptographiques (22 contrôles).
- `node scripts/e2ee-bench.mjs` : banc de calcul ; résultats bruts dans docs/memoire/bench-e2ee.json.
- `npx tsc --noEmit`, `npx eslint src`, `npx next build` : vérifications statiques et compilation.

Annexe E — Chronologie du dépôt

Tableau 21 — Jalons du dépôt (34 commits entre le 21 et le 25 septembre 2026)

Date	Jalon (messages de commit résumés)
21/09	Application PWA bilingue ; espaces personnel, Refuge et couple ; interface en verre dépoli ; tests de bout en bout
23/09	Messagerie (cadre sous clavier, appui long, surnoms) ; appels audio et vidéo WebRTC ; remplacement du service d'appel payant par TURN Cloudflare / coturn ; Web Push ; verrou d'application ; correctifs vocaux, haut-parleur, latence ; photos enregistrables ; tonalités
25/09	Persistance et palette d'appel ; historique et rappel ; chiffrement de bout en bout ; émojis et réactions par sticker ; recherche et liens ; correctifs de mise en page